

UNIVERSIDAD POLITÉCNICA DE MADRID
ESCUELA TÉCNICA SUPERIOR DE INGENIEROS INDUSTRIALES
CENTRO DE ELECTRÓNICA INDUSTRIAL

TRABAJO DE FIN DE GRADO

Metodología de diseño digital para FPGAs desde lenguaje C orientada al “Do It Yourself”

Autora: Raquel Buezo Tordesillas

Directores: Yago Torroja
Jorge Portilla



Madrid, Julio 2016

Agradecimientos

Considero que no hay mejor manera de comenzar este trabajo que agradeciendo a aquellos que me han acompañado y ayudado a lo largo de estos meses.

En primer lugar agradecer a mis tutores, Yago Torroja y Jorge Portilla, por su paciencia, por todo el conocimiento que me han aportado, por enseñarme a afrontar los problemas desde otro punto de vista, pero sobre todo, por darme el impulso y la fuerza que algunas veces me han faltado en estos meses.

A mis padres, Ester y Jorge, por su apoyo incondicional y por soportarme en mis peores momentos. Por nunca dudar de mí y creer, aun cuando ni yo lo creía, que era capaz de terminar. Ellos me han enseñado que con esfuerzo y ganas todo se consigue, y gracias a ellos me he convertido en la persona que soy hoy.

También quiero agradecer a mi hermana, Marina, por hacerme mantener la cabeza fría en mis momentos de debilidad, por aguantar lo agonías que soy a veces y estar ahí en cada paso del camino.

A mis amigas, las que siempre están en los momentos buenos y especialmente en los malos, para levantarme tantas veces como caiga. Por no dejarme flojear y enseñarme que las cosas no son siempre blancas o negras.

Por último dar las gracias a mis compañeros durante estos cuatro años de carrera, con mención especial a Carlos y a Eduardo, compañeros sin los que no habría sobrevivido a esta especialidad. Por haberse convertido en los mejores amigos dentro y fuera de esta universidad, y a los que solo les ha faltado poner su cama en el departamento de electrónica junto a la mía. Gracias por las risas, las cervezas, y por enseñarme que de la carrera, además de formación, me llevo unos grandes amigos.

A todos vosotros, MUCHAS GRACIAS

Raquel

Resumen Ejecutivo

El presente Trabajo Fin de Grado es una primera aproximación dentro de un proyecto a más largo plazo en el que se pretende estudiar una metodología para facilitar el diseño de circuitos digitales de mediana complejidad a individuos con cierta experiencia en programación *software*. Para ello se creará un sencillo entorno de desarrollo que mediante el empleo del lenguaje C, permitirá a los usuarios realizar diseños digitales para ser integrados dentro de una arquitectura constituida por un microcontrolador y una FPGA.

Una FPGA es un dispositivo *hardware* reconfigurable compuesto de celdas lógicas, con el objetivo de realizar diseños digitales concretos. Esto permite implementar de manera flexible un sistema específico para cada tarea y al mismo tiempo ofrecer todas las ventajas del *hardware* en cuanto a unos niveles de eficiencia superiores a los que se podría llegar con un microprocesador. Gracias a los diseños *hardware* optimizados que se consigue en estos dispositivos, se logra acelerar considerablemente la ejecución además de reducir el consumo.

Sin embargo, la mayor desventaja de las FPGAs radica en la fase de descripción para realizar un determinado diseño. Esto se debe a la dificultad intrínseca de los circuitos digitales y a la necesidad de emplear lenguajes de descripción *hardware* (HDL por sus siglas en inglés, *Hardware Description Language*). Estos lenguajes, aunque formalmente similares a los lenguajes de programación *software*, presentan un nivel de dificultad mucho mayor, principalmente debido a la existencia de sentencias concurrentes y sentencias secuenciales que requieren conocimientos *hardware* para estructurar bien el diseño. Actualmente, existen muchos menos desarrolladores *hardware* que programadores *software*, por lo que se está infrutilizando el potencial de los dispositivos tipo FPGA.

Por otro lado, iniciativas como el ecosistema Arduino han mostrado que es posible reducir la complejidad de los procesos de diseño si se disponen de las herramientas y la metodología adecuadas. En este sentido, y en lo que al desarrollo del *software* y sistemas empuados se refiere, Arduino ha “democratizado” el acceso al diseño de sistemas embebidos, y lo que hace unos pocos años era ámbito casi exclusivo de universidades y empresas es ahora accesible por usuarios sin el mismo nivel de formación.

Sin embargo, esta democratización en el desarrollo *software* no ha tenido su paralelo en el desarrollo de diseños digitales directamente en *hardware*. Por ello, surge la idea de la creación de una plataforma que facilite el diseño de *hardware*, aprovechando para ello el uso de lenguajes ya conocidos por este tipo de usuarios, como es el lenguaje C. La concepción global (Figura 0.1) consiste en la creación de un entorno de desarrollo, un preprocesador y una herramienta de síntesis, posibilitando que el usuario describa funcionalidades *hardware* de forma sencilla mediante lenguaje C, que posteriormente serán traducidas mediante herramientas de síntesis digital, e integradas de forma casi transparente para el usuario dentro de una arquitectura que incluye un microprocesador y los módulos

hardware desarrollados por el usuario (similar a un microcontrolador con periféricos no genéricos, sino diseñados para la aplicación).

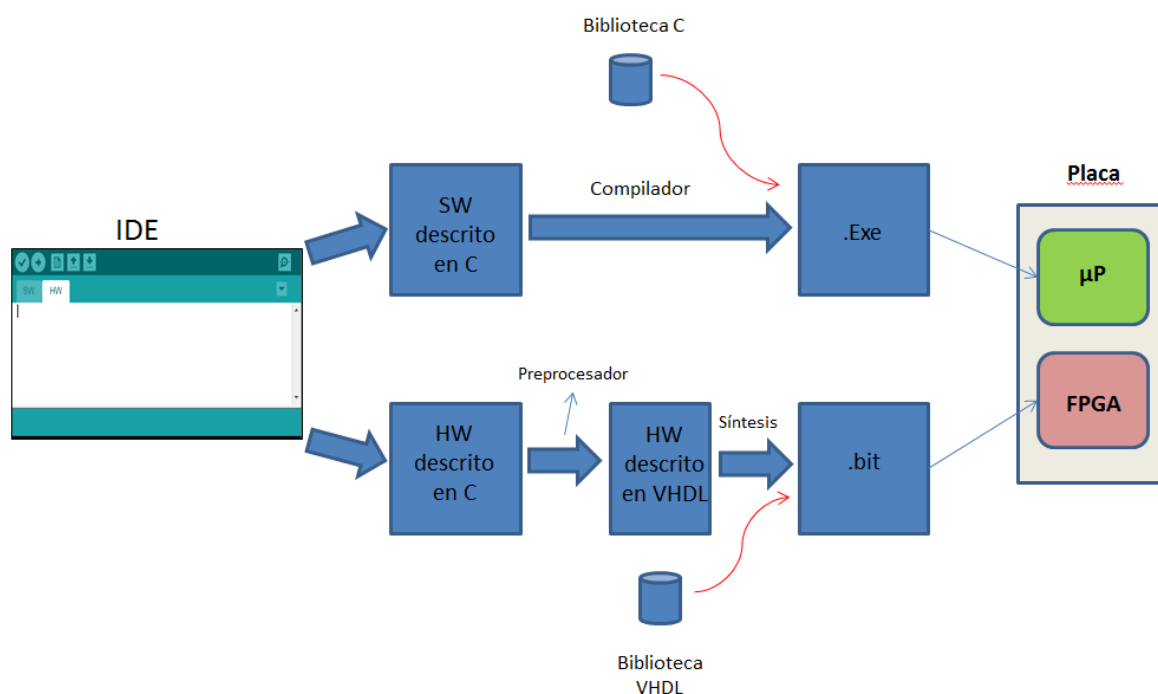


Figura 0.1- Planteamiento del flujo de diseño de microprocesador y FPGA integrados en una misma placa. (Fuente: Autoría propia)

Para alcanzar el objetivo propuesto, el desarrollo del trabajo se ha dividido en varias fases:

La **primera fase** se centró en la investigación del ámbito “Do It Yourself” en el que se impulsan proyectos de todo tipo para ayudar a principiantes a adquirir conocimientos de una gran variedad de disciplinas y a aplicarlos en áreas diversas. Después de un análisis de las características de los proyectos existentes en el campo de la electrónica se destacó Arduino para tomarlo como referencia en el desarrollo del proyecto.

La decisión de fijarse en Arduino fue debida a su éxito como proyecto en la comunidad DIY, que ha ayudado a mejorar el interés para crear y diseñar, y la capacidad para resolver problemas en los distintos niveles de educación. De Arduino hay que destacar su sencillo entorno de desarrollo (IDE), su ventajosa biblioteca de funciones, y su excelente metodología docente, que en conjunto han logrado simplificar el diseño de sistemas empotrados para usuarios noveles.

En la **segunda fase**, se diseñaron dos módulos y una interfaz genérica en VHDL (el lenguaje de descripción *hardware* elegido para este proyecto), para ser implementados en la FPGA de la placa de ensayos.

La necesidad de crear una interfaz surge por la existencia de solamente dos puertos bidireccionales (de 8 bits) entre el microcontrolador y la FPGA de la placa de ensayos (Figura 0.2). Es por eso que se utiliza uno de los puertos como bus multiplexado de datos y direcciones, y el segundo puerto para mandar las señales de control desde el microprocesador a la FPGA que se considera siempre “esclava”.

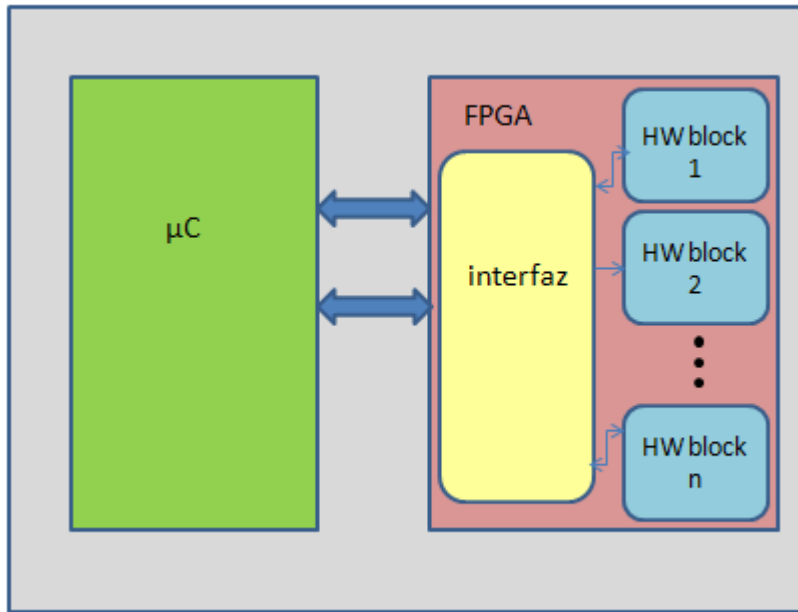


Figura 0.2- Esquema general de placa con microcontrolador y FPGA. (Fuente: Autoría propia)

En la interfaz genérica se define entonces el protocolo empleado para la comunicación entre el microprocesador y la FPGA integrados en la misma placa. Se tienen en cuenta los cuatro posibles escenarios de comunicación entre los dos dispositivos, que permiten la escritura y lectura de datos y direcciones. Para decidir en qué escenario está, la interfaz interpreta las señales de control procedentes del microprocesador y en función de estas, lee o escribe un dato o una dirección sobre el bus de datos y direcciones.

En cuanto al diseño de los módulos *hardware*, se eligió llevar a cabo un bloque para generar señales PWM y un bloque para controlar el funcionamiento de tiras de LEDs. La decisión tomada sobre la elección de estos bloques concretos, fue debida al nivel de complejidad medio que implica su diseño en VHDL y de cara al futuro intento de descripción mediante un subconjunto de lenguaje C.

Esta fase sirvió de aprendizaje del lenguaje de descripción *hardware* y sus dificultades, además de para crear dos módulos “tipo” útiles.

Finalmente, en la **tercera fase** se ha planteado una primera propuesta para la descripción *hardware* mediante un subconjunto de datos y estructuras de C. En esta propuesta se expone un *template* a seguir en la descripción *hardware* mediante el uso de lenguaje C. En

este *template*, primero se declaran las variables globales que tras la síntesis, generarán registros (en el diseño digital) a los que se tendrá acceso desde el programa. Después se declaran las variables locales del bloque, que serán justamente las variables locales del proceso en VHDL de ese módulo. Además, existirán dos funciones comunes a todas las descripciones; la primera será el *SetupHardware* donde se realice la asignación de las distintas variables a los pines de entrada/salida de la placa. Y la segunda será la función *HardwareLoop* donde se describirá la funcionalidad del diseño mediante el subconjunto especificado de datos y estructuras de C.

Para probar la viabilidad de la propuesta, se han descrito en C los mismos módulos realizados en VHDL en la segunda fase, y se ha conseguido demostrar la sencillez de la programación en C (bajo la arquitectura estándar tipo microcontrolador) frente a la descripción *hardware* con VHDL.

PALABRAS CLAVE

FPGA, Hardware Description Language, Do It Yourself, microprocesador, bus, registro, *hardware*, *software*

CÓDIGOS UNESCO

330404 - Unidades Centrales de Proceso

330406 - Arquitectura de ordenadores

330412 - Dispositivos de control

330416 - Diseño lógico

330499 - Síntesis de Alto Nivel

330499 - Sistemas reconfigurables basados en FPGA

Índice General

1. Introducción.....	9
1.1. Motivación	10
1.2. “Do It Yourself”	10
1.2.1. Arduino	12
1.2.2. Mojo.....	13
2. Objetivos.....	17
3. Marco teórico y estado del arte	21
3.1. Field Programmable Gate Arrays.....	21
3.1.1. Definición	21
3.1.2. Historia de las FPGAs.....	22
3.2. Lenguajes de descripción <i>hardware</i>	23
3.3. FPGAs y la programación <i>software</i>	23
3.3.1. Flujo de diseño para FPGAs	24
3.3.2. Síntesis de alto nivel	25
4. Desarrollo e Implementación	29
4.1. Diseño de módulos <i>hardware</i>	29
4.1.1. PWM.....	29
4.1.2. LEDS	32
4.1.3. Interfaz	37
4.2. Simulación e implementación	41
4.2.1. Simulaciones.....	41
4.2.2. Implementación	44
5. Propuesta de descripción <i>hardware</i> mediante C	49
5.1. Subconjunto de C y método de traducción a VHDL.....	49
5.2. Ejemplos de aplicación.....	54
5.3. Emulación <i>software</i>	56
6. Conclusiones y líneas futuras	59
6.1. Conclusiones	59
6.2. Líneas Futuras	61
7. Organización del TFG	65

7.1.	Planificación temporal.....	65
7.1.1.	Estructura de Descomposición del Trabajo (EDP)	65
7.1.2.	Diagrama de Gantt	66
7.2.	Presupuesto.....	67
Referencias Bibliográficas.....		69
Índice de Figuras.....		71
Índice de Tablas		72
Abreviaturas y Acrónimos		73

1. Introducción

1. Introducción

En nuestros días, el mundo está viviendo una era donde la dependencia de la tecnología y de los dispositivos electrónicos es cada vez mayor. Debido a la necesidad de procesar cada vez más información, existe mucha presión en el desarrollo de microprocesadores cada vez más potentes y rápidos. Sin embargo la evolución y mejora de estos dispositivos se ha visto limitada por ciertos impedimentos, entre los cuales destaca la imposibilidad de conseguir un mayor aumento de la frecuencia.

Como alternativa, se están realizando desarrollos de nuevos modelos de computación basados en la tecnología disponible y en la posibilidad de paralelizar la ejecución de ciertas tareas.

Una posible aproximación se basa en la creación de dispositivos con varios núcleos de procesamiento, conocidos como microprocesadores multinúcleo (*multicore*) en los cuales un solo procesador tiene capacidad para procesar en varios núcleos distintos en paralelo.

Sin embargo si se trata de ejecución en paralelo, siempre una solución puramente *hardware* parece más adecuada y de ahí que la solución a este problema mediante el uso de FPGAs (*Field Programmable Gate Arrays*) ofrece un potencial mucho mayor y cada día está siendo adoptada de manera más genérica.

Una FPGA es un dispositivo *hardware* compuesto de elementos lógicos que está preparado para ser configurado después de su fabricación, con el objetivo de realizar aplicaciones específicas. Esto permite implementar de manera flexible un sistema específico para cada tarea y al mismo tiempo ofrecer todas las ventajas del *hardware* en cuanto a unos niveles de eficiencia superiores a los que se podría llegar con un microprocesador.

Estos dispositivos *hardware* permiten liberar al microprocesador de la carga de ejecutar muchas tareas que pueden ser realizadas en paralelo. De esta manera se consigue acelerar considerablemente la ejecución y al mismo tiempo reducir el consumo, pues parte de las tareas corren directamente en *hardware* y se crean diseños optimizados para las mismas.

Actualmente, el diseño de las FPGAs requiere el empleo de lenguajes de descripción *hardware* (HDLs, *Hardware Description Languages*), que son lenguajes complejos de utilizar y que requieren de conocimientos específicos de *hardware*.

Debido a la complejidad de los HDLs y a la falta de experiencia de los usuarios con el diseño *hardware*, se está infrautilizando el enorme potencial que ofrece el uso de las FPGAs. Esto manifiesta una clara necesidad de estudiar la forma de facilitar el uso de estas tecnologías al público.

1.1. Motivación

Hoy en día existen una gran variedad de proyectos basados en la filosofía del “Hágalo Usted Mismo”, (DIY o “*Do It Yourself*”). Este término hace referencia a la realización por uno mismo de aparatos u objetos que integran distintos tipos de tecnología y otros aspectos artísticos y funcionales. Pero hoy, el DIY no sólo se enfoca hacia la producción de objetos, sino hacia el propio proceso de aprendizaje. Así, DIY, Internet, creatividad y autoaprendizaje se están estableciendo como un nuevo paradigma de formación, muy adecuado a una demanda cada vez mayor de profesionales flexibles que se adapten rápidamente a las condiciones cambiantes del mercado. Por todo ello, se impulsan proyectos de todo tipo para ayudar a principiantes a adquirir conocimientos básicos y aplicarlos en una gran variedad de campos.

Basándose en esta idea, y en la dificultad en el diseño *hardware* de las FPGAs expuesto anteriormente, en este proyecto se quiere usar el enfoque DIY para simplificar el trabajo con FPGAs, facilitando el acceso del público a esta tecnología. Se pretende estudiar posibles técnicas de acercamiento al gran público de esta tecnología sin necesidad de ser un experto en diseño digital ni en electrónica (al igual que Arduino ha conseguido con la programación y uso de sistemas embebidos). Poniéndolo en lenguaje llano, se pretende llegar a hacer “el Arduino de las FPGAs”. En el presente proyecto se aborda parte del problema, centrándose principalmente en estudiar la viabilidad del diseño de módulos para FPGAs mediante el uso de un subconjunto reducido de lenguajes tipo C.

1.2. “Do It Yourself”

Las herramientas de aprendizaje han evolucionado continuamente desde la pizarra y tiza a utilizar las nuevas tecnologías como formación online o programas de asistencia. “Do It Yourself” es un paso más y es el método de construir, modificar o reparar algo sin la ayuda directa de expertos o profesionales [1].

Como se mencionó anteriormente, los dispositivos electrónicos y las herramientas para su diseño han sufrido una enorme evolución en los últimos años. Debido a esto, han aparecido un gran número de proyectos DIY para facilitar el aprendizaje y el uso de estos dispositivos.

En la investigación realizada sobre los distintos proyectos DIY [2, 3, 4, 5, 6, 7] en el ámbito de la electrónica y la programación, se han estudiado una serie de categorías clave para evaluar la plataforma más adecuada para este desarrollo. Los aspectos analizados son, entre otros, si los proyectos están enfocados hacia el aprendizaje de *hardware* o *software*, cuál es el lenguaje de programación que emplean, si contienen sus propias bibliotecas de código y si es necesario descargar su entorno de desarrollo.

A continuación se muestra en la Tabla 1.1 la recopilación de los resultados destacados del análisis.

	Aprendizaje HW/SW	Lenguaje de programación	Interfaz propia	Bibliotecas propias	Dispositivo
Arduino	▪ SW	▪ C/C++	✓	✓	▪ Single board microcontroller
Fritzing			✓	✓	▪ PCB (Printed Circuit Boards)
Papilio			✓	✓	▪ FPAG circuits
Mojo	▪ HW	▪ Verilog, Lucid	✓	✓	▪ FPGA
FPGA4FUN	▪ HW	▪ Verilog			▪ FPGA
Raspberry Pi	▪ SW	▪ Scratch, Python	✓		▪ Credit-card sized computer

Tabla 1.1- Proyectos DIY destacados en el ámbito de la electrónica (Fuente: Autoría Propia)

Es importante resaltar que el líder en proyectos DIY sobre electrónica en el mercado actual es Arduino. Los aspectos claves de Arduino son el tener su propio entorno de desarrollo, sencillo e intuitivo para el usuario, que permite escribir código en lenguaje C utilizando una biblioteca de funciones muy simplificada. El código escrito puede usarse para programar los microprocesadores de cualquier placa Arduino, que disponen de varias entradas y salidas para interactuar con sensores y actuadores de todo tipo (luces, motores, etc.) [3].

La introducción de Arduino en los distintos niveles de educación ha ayudado a mejorar el interés para crear y diseñar, y a mejorar el pensamiento crítico y la capacidad para resolver problemas. Hay muchos proyectos que se pueden encontrar en la red y cada día se desarrollan muchos más.

Aunque hay otras plataformas similares de código/*hardware* abierto, se ha seleccionado Arduino por las múltiples ventajas que ofrece, fácil de utilizar, asequible, disponible y versátil. Al mismo tiempo, es un entorno que usa la interacción con elementos eléctricos y sensores como motor para explorar y aprender técnicas de programación.

Por otro lado, hay que hacer referencia al proyecto Mojo, ya que sus objetivos son los que más se asemejan a los de este TFG. Las similitudes van desde el dispositivo sobre el que se quiere formar a los usuarios, las FPGAs, hasta la idea de la creación de un nuevo lenguaje para el diseño de estos dispositivos que sea sencillo de aprender para los no expertos en diseño digital.

1.2.1.Arduino

La compañía Arduino nació en el Ivrea Interaction Design Institute, de donde surgió la idea de diseñar una herramienta para el prototipado rápido, dirigido hacia estudiantes sin una buena formación en electrónica y programación [3].

Sin embargo, su posterior evolución lo han llevado a convertirse en uno de los proyectos líderes en el ámbito “Do It Yourself”, cuyo principal enfoque es acercar y facilitar el uso de la electrónica y la programación de sistemas embebidos en aplicaciones multidisciplinarias.

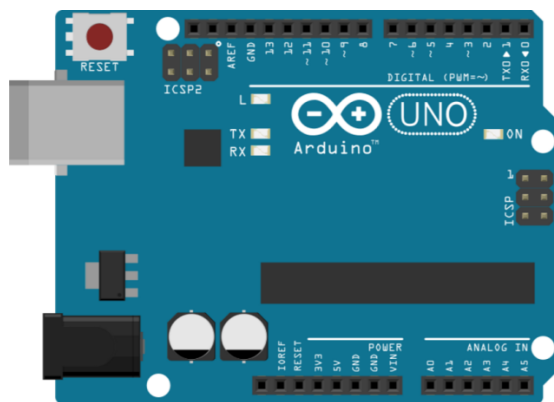


Figura 1.1- Placa Arduino UNO. (Fuente: [3])

Arduino es una comunidad tecnológica que diseña y manufactura placas (Figura 1.1) orientadas al desarrollo de *hardware* y *software*, y que cuenta además con su propio entorno de desarrollo (IDE, *Integrated Development Environment*). El lenguaje es de C/C++, facilitando su uso a través de su IDE y de una biblioteca de funciones sencilla. Todo ello no impide que pueda ser programado directamente en ensamblador o en C/C++ puro, usando otras plataformas de desarrollo [3].

Arduino es además una plataforma *open source* y *open hardware* que permite libertad de acceso a sus componentes de *hardware* así como al *software*. *Open source* es una expresión inglesa que pertenece al ámbito de la informática. Se califica como *open source* [8] a los programas informáticos que permiten el acceso a su código fuente; esto facilita que programadores ajenos puedan realizar modificaciones sobre el programa.

Arduino simplifica el proceso de trabajar con microprocesadores, ofreciendo ventajas respecto a otros sistemas entre las que se pueden destacar las siguientes [3]:

- Existencia de una **comunidad** de usuarios/miembros activos. La comunidad consiste en grupos de personas que se ayudan mutuamente a resolver dudas y problemas así como a contribuir al futuro del proyecto con posibles ideas y mejoras.
- **Hardware económico**. Las placas Arduino son relativamente baratas comparadas con las de plataformas similares.

- **Multi-plataforma.** El *software* de Arduino (IDE) es compatible con los sistemas operativos de Windows, Macintosh OSX y Linux.
- **Entorno de programación** sencillo y claro. El *software* de Arduino (IDE) es fácil de utilizar para principiantes pero también muy flexible para usuarios avanzados. Es utilizado tanto por profesores como alumnos, diseñadores, artistas o cualquier aficionado con interés en la materia.
- Programado vía cable **USB**. No existe necesidad de componentes especiales para operar con las placas Arduino, haciéndolo así más accesible.

Gracias a una experiencia de usuario sencilla y accesible, Arduino es utilizado en miles de aplicaciones hoy en día en distintas partes del mundo.

1.2.2. Mojo

La empresa Embedded Micro [5] se ha dedicado a facilitar el uso de las FPGAs al público sin necesidad de formación previa. Su proyecto consiste en la creación de un entorno de desarrollo llamado Mojo IDE (cuyas características se expondrán en este apartado) y que tiene como objetivo facilitar la programación de su placa Mojo.

Además, para facilitar la programación de su placa han desarrollado un nuevo lenguaje de descripción *hardware* llamado Lucid, que se centra en reducir la cantidad de código necesaria para describir diseños simples y fácilmente comprensibles.

Este lenguaje de descripción *hardware* está basado principalmente en Verilog, uno de los lenguajes *hardware* más comúnmente empleados, y tiene muchos conceptos similares a VHDL. Pero a su vez trata de asemejarse a la sintaxis de C/C++ para aprovechar los conocimientos de muchos programadores *software*.

MOJO IDE

El Mojo Integrated Development Environment (IDE) creado por Embedded Micro se apoya en ISE (*Integrated Software Environment*) de Xilinx para la síntesis del proyecto.

El IDE se utiliza para la programación de la FPGA de las placas también diseñadas y desarrolladas por esta misma empresa. La última versión es la placa Mojo V3 (Figura 1.2) que incluye una FPGA Spartan 6 XC6SLX9, un microcontrolador ATmega32U4, pines digitales IO, pines de entrada analógicos, 8 LEDS, y un *bootloader* compatible con Arduino para facilitar la programación del microcontrolador [5].

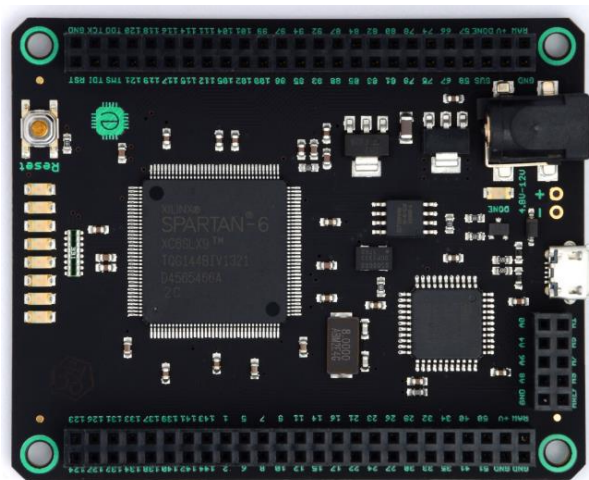


Figura 1.2- Placa Mojo V3 (Fuente: [5])

El entorno de desarrollo se muestra en la Figura 1.3 y consiste en un editor de texto en el que se puede programar tanto en Lucid como en Verilog y además se pueden añadir tantas pestañas en el editor de texto como módulos *hardware* se quieran describir.

Todos los proyectos creados en el IDE tienen un archivo “*top*” que contiene las entradas y salidas externas del diseño descrito. Si se añaden más módulos al proyecto, estos deben ser sub-módulos del “*top-module*”. La jerarquía entre ellos se puede apreciar en el lateral izquierdo del IDE donde se muestran todos los módulos pertenecientes al proyecto.

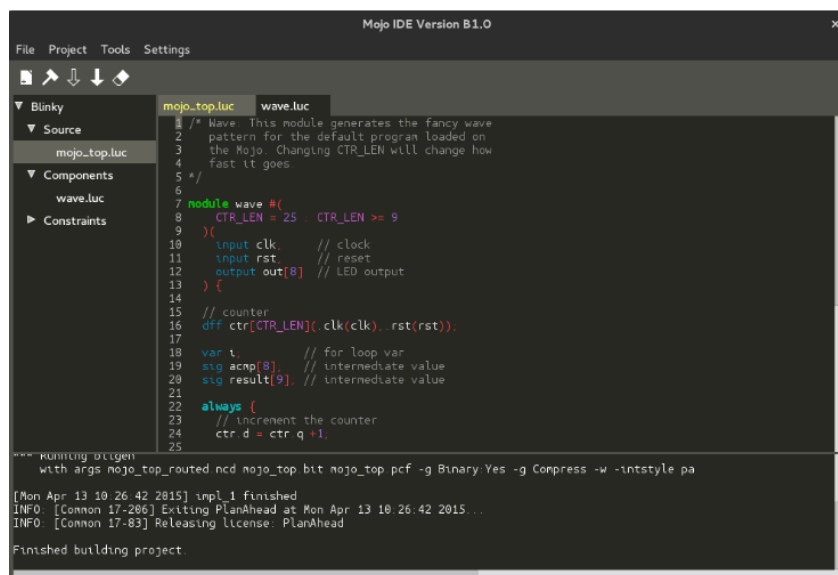


Figura 1.3- Mojo IDE (Fuente: [5])

2. Objetivos

2. Objetivos

El objetivo principal de este proyecto es plantear procedimientos que faciliten el trabajo con FPGAs, frente al uso de sistemas que sólo disponen de microcontroladores. Para ello se pretende usar enfoques de alto nivel que aprovechen los conocimientos de programación de los usuarios para facilitar el trabajo con esta tecnología.

En este proyecto se hace uso de una placa diseñada y creada en la División de Ingeniería Electrónica (DIE) de la Escuela Técnica Superior de Ingenieros Industriales. En dicha placa se han realizado las pruebas de los elementos diseñados, tanto los bloques *hardware* descritos en VHDL como los elementos *hardware* que permiten acceder a ellos y configurarlos.

Este Trabajo Fin de Grado se considera como una primera aproximación dentro de un proyecto a largo plazo, cuyo objetivo es la creación de una plataforma de diseño de *hardware* mediante el uso de lenguajes C. La idea final consiste en la creación de un entorno de desarrollo y de un conjunto de programas “ejemplo” que permitan a los usuarios aprender a diseñar distintos bloques *hardware* que serán implementados en la FPGA.

La finalidad es conseguir que el usuario pueda escribir de forma sencilla el código que debe ejecutar el microprocesador y el *hardware* de la FPGA, integrados en una misma placa (Figura 2.1), sin necesidad de atender a los detalles de la interfaz entre ellos y las dificultades que esto conlleva.

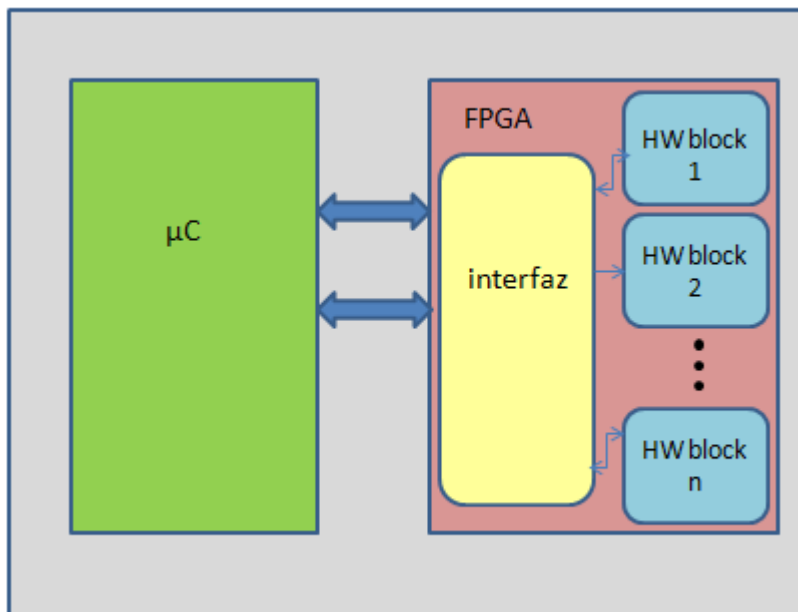


Figura 2.1- Esquema general de una placa con microprocesador y FPGA. (Fuente: Autoría propia)

A lo largo del documento se explicarán en más detalle cada uno de los objetivos expuestos a continuación y se hablará del desarrollo realizado. Se expondrá una valoración de los objetivos en el capítulo 6 (Conclusiones y líneas futuras), mencionando también posibles mejoras y futuras líneas de trabajo.

Los objetivos marcados para este proyecto son:

1. Investigación en el ámbito “Do It Yourself” de los proyectos ya existentes en el campo de introducción a la electrónica.
2. Diseño de dos módulos “tipo”, descritos en el lenguaje de descripción *hardware* VHDL, para ser implementados en una FPGA.
3. Desarrollo de una interfaz genérica entre microprocesador y FPGA integrados en una misma placa.
4. Aplicación práctica para comprobar el correcto funcionamiento del sistema en la placa diseñada en el DIE que se mencionó al comienzo de este capítulo.
5. Planteamiento de una posible y futura traducción de VHDL a C para facilitar el uso de las FPGAs a usuarios sin experiencia en programación de *hardware*.

3. Marco teórico y estado del arte

3. Marco teórico y estado del arte

El presente proyecto se basa en el estudio de una metodología de diseño digital para FPGAs orientadas al “Do It Yourself”. Para el desarrollo y las pruebas se ha usado una placa creada y desarrollada en la División de Ingeniería Eléctrica de la Escuela Superior de Ingenieros Industriales.

En este capítulo se explican los principales conceptos teóricos relacionados con el proyecto para facilitar su comprensión, y un breve análisis del estado presente del desarrollo de temas relacionados.

3.1. Field Programmable Gate Arrays

3.1.1. Definición

Una FPGA es un circuito digital integrado el cual puede ser configurado por un usuario o diseñador. En general se utiliza un lenguaje de descripción *hardware* (HDL, *Hardware Description Language*) para describir la configuración deseada.

Una FPGA puede ser utilizada para implementar cualquier función lógica. Estos dispositivos se han hecho verdaderamente populares debido a la facilidad para revisar la funcionalidad incluso después de ser implementada, a la reconfiguración como parte del diseño y a los bajos costes de ingeniería.

Las FPGAs se basan en ciertos bloques lógicos configurables (CLBs, *Configurable Logic Blocks*) y conexiones también configurables entre ellos, parecido a como se conectan las puertas lógicas en las diferentes configuraciones (Figura 3.1). Estos bloques lógicos pueden ejecutar funciones tan básicas como una puerta AND o también combinaciones lógicas muy complejas. En prácticamente todas las FPGAs, los bloques lógicos también incluyen elementos de memoria que pueden ser desde *flip-flops* (registros) básicos a bloques de memoria completos [9].

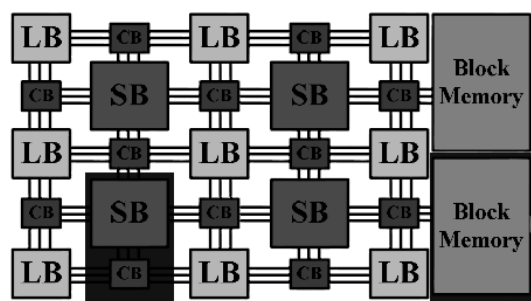


Figura 3.1-Bloques lógicos configurables de una FPGA. (Fuente: [10])

3.1.2. Historia de las FPGAs

Xilinx introdujo la primera FPGA en 1984 [11], aunque no llegó a llamarse FPGA hasta que en 1988 Actel popularizó dicho nombre.

Desde su introducción, estos dispositivos han progresado a través de distintas fases de desarrollo. Cada fase era motivada a la vez por una combinación de avances en la tecnología (como por ejemplo la fuerte influencia de la evolución de los semiconductores), oportunidad y una fuerte demanda de las aplicaciones. Todos estos factores externos han creado a lo largo del tiempo una presión que ha causado los cambios en las distintas características de los dispositivos y de las herramientas para su diseño [11].

Se puede observar en la Figura 3.2 el crecimiento en capacidad y rendimiento de las FPGAs, al mismo tiempo que la enorme reducción en coste y consumo desde que fueron introducidas en el mercado hasta la actualidad.

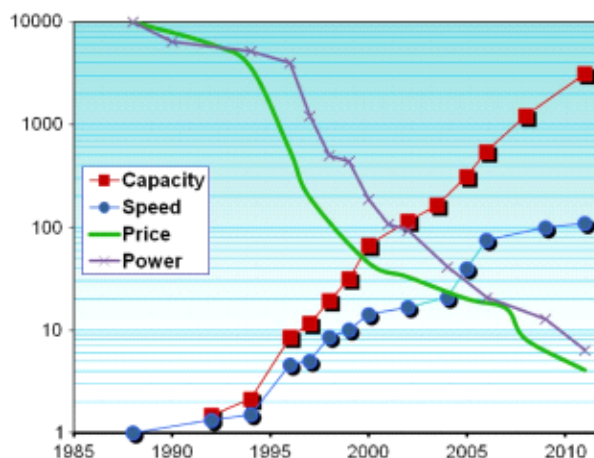


Figura 3.2- Evolución de las características de las FPGAs. (Fuente: [11])

El diseño de las FPGAs originalmente se hacía de manera similar a como se hacía con los chips lógicos discretos mediante herramientas esquemáticas que mostraban elementos y puertas lógicas unidas unas con otras [11]. Pero según crecía la capacidad de las FPGAs, también crecía la complejidad de los diseños que se podían acometer. Esto generó una evolución de las herramientas de diseño para aprovechar dicho potencial.

Hoy en día, de entre todas las herramientas que se fueron creando, el lenguaje de descripción *hardware* (HDL) ha quedado como la solución de facto sobre cualquier otra. Dentro de los HDLs, los dos más comúnmente empleados para el diseño de sistemas *hardware* de más alto nivel son Verilog y VHDL.

3.2. Lenguajes de descripción *hardware*

Diseñar un circuito para una FPGA requiere el uso de un lenguaje de descripción *hardware* (HDL) para describir la configuración deseada. Aunque formalmente los HDL son parecidos a los lenguajes de programación de *software* son muy diferentes en cuanto a su interpretación. Así, mientras un programa descrito en C consiste de una serie de instrucciones que se describen y ejecutan de forma secuencial, una descripción en HDL describe circuitos concurrentes trabajando en paralelo y al contrario que uno escrito en C, debe complicarse mucho para conseguir funcionalidades secuenciales. Son dos enfoques distintos y es complejo diseñar un circuito digital mediante un HDL si se parte de una mentalidad de desarrollador de *software*.

Como ya se mencionó en el apartado anterior, los dos HDLs más comúnmente empleados son Verilog y VHDL. En la etapa de diseño de este proyecto se ha empleado VHDL para diseñar los módulos *hardware*.

Estos conceptos, que son parte de la semántica del lenguaje de VHDL, permiten a los desarrolladores de un cierto nivel escribir rápidamente la descripción de circuitos complejos con un esquema relativamente sencillo.

Una característica a destacar de un diseño en VHDL es su orden jerárquico de módulos. Los módulos resumen la jerarquía del diseño y se corresponden con otros bloques a través de un conjunto de puertos declarados [9].

En conclusión, los lenguajes de descripción *hardware* ofrecen grandes prestaciones para el diseño de FPGAs y otros dispositivos *hardware*, pero desafortunadamente están siendo infrautilizados debido a su dificultad y a la falta de conocimiento en este campo.

3.3. FPGAs y la programación *software*

Aunque existen muchos desarrolladores *software*, hay muchos menos diseñadores de *hardware*, y más en concreto de FPGAs. Como se mencionó anteriormente, uno de los objetivos de este trabajo es facilitar el diseño y programación de las FPGAs como método de aprendizaje de electrónica digital.

Ha habido quien ya ha tratado de abordar este problema y ayudar a hacer más accesibles las FPGAs al mundo de los ingenieros *software*. Se ha intentado demostrar que las FPGAs no son solo una alternativa para ingenieros de desarrollo *hardware* con un gran nivel de experiencia, sino que hay otras opciones para que los ingenieros *software* también se puedan aprovechar del rendimiento y la eficiencia de las FPGAs.

Que un dispositivo sea programable, significa que permite su adaptación para ejecutar diferentes tareas utilizando el mismo *hardware* físico, independiente de si hablamos de una CPU o una FPGA [12]. Esto es muy útil para compartir recursos entre distintas tareas y por

tanto para reutilizar los recursos a lo largo del tiempo, pues se podría utilizar el mismo dispositivo para varias aplicaciones distintas. Esto tiene un impacto importante en el coste de los dispositivos. El poder utilizar el mismo dispositivo para distintas aplicaciones y evoluciones de la misma en el tiempo, hace que crezca el número de unidades necesarias de un determinado dispositivo y se pueda reducir el coste de los mismos de manera significativa al poder producirse en grandes volúmenes.

Una gran diferencia entre las plataformas de programación *hardware* y *software* radica en la posibilidad de controlar la estructura y la arquitectura en el caso de diseño *hardware*. Cuando se desarrolla un algoritmo en *software* para una CPU, hay que preocuparse de los requisitos de tiempo de ejecución además de las necesidades de memoria. En el caso de las FPGAs, no sólo están vinculadas a un dominio de tiempo, sino que también lo están a un dominio espacial, lo que significa que hay que tener en cuenta el espacio físico que puede requerir un cierto algoritmo o función lógica [12].

Para las FPGAs, las únicas limitaciones son la disponibilidad de recursos y la frecuencia de reloj a la que puede trabajar la funcionalidad descrita. Con los dispositivos que existen hoy en día, que contienen millones de celdas lógicas, miles de bloques aritméticos y decenas de megabytes de memoria interna, esta limitación impone pocas restricciones, para diseños de complejidad media y media-alta.

3.3.1. Flujo de diseño para FPGAs

El flujo de diseño para FPGAs comprende distintos niveles de abstracción que representan la descripción sobre múltiples pasos de transformación con el objetivo final de configurar el *bitstream* para ser programado en la FPGA [12].

El nivel de abstracción RTL (Register Transfer Level) es básicamente una abstracción donde toda la información de estado es almacenada en registros (o cualquier clase de memoria) y donde la lógica (la cual incluye aritmética) entre los registros es utilizada para generar o calcular nuevos estados [12]. Por lo tanto, el nivel RTL describe todos los elementos de memoria (por ejemplo flip-flops, registros o memorias) y la lógica utilizada además de las conexiones entre los diferentes elementos y por tanto el flujo de datos a través de un circuito.

Las especificaciones RTL son comúnmente realizadas utilizando lenguajes de descripción *hardware* (HDLs). Estos lenguajes proporcionan tipos de datos, operaciones lógicas y aritméticas, jerarquías y muchas construcciones de programación (por ejemplo bucles) que son conocidos para lenguajes de programación *software* como C. Sin embargo, el *hardware* es paralelo de forma innata, así que las instrucciones no son ejecutadas de manera secuencial sino que muchos módulos se ejecutarán concurrentemente en el sistema, lo cual no es soportado por los lenguajes de programación *software*.

El *Register Transfer Level* es bastante bajo y en una analogía al *software*, se puede decir que es comparable a una abstracción en el rango cercano entre ensamblador y C.

Para obviar todo este nivel de abstracción bajo, se han creado las herramientas de síntesis a alto nivel que permiten a los diseñadores *hardware* realizar una descripción a alto nivel, dejando que se encargue el sintetizador de la posterior transformación a RTL [12].

Un diagrama de flujo genérico para el diseño de FPGAs incluyendo los pasos y distintos niveles de abstracción, se muestra en la Figura 3.3.

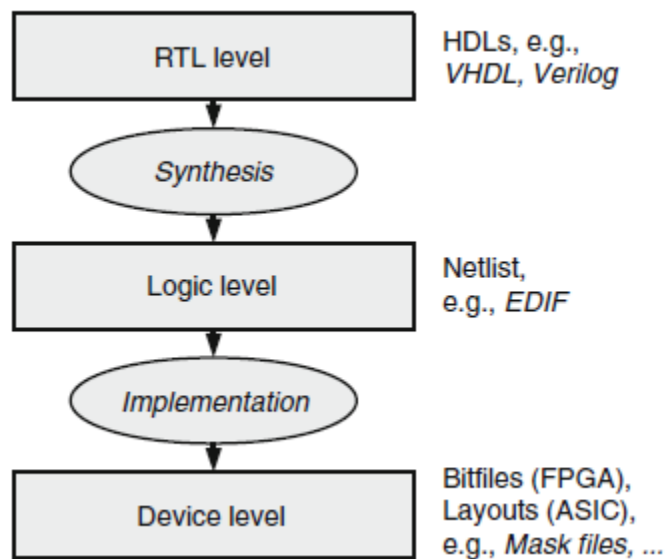


Figura 3.3-Flujo de diseño de una FPGA. (Fuente: [12])

3.3.2. Síntesis de alto nivel

Como ya se ha mencionado, en la automatización de diseño electrónico (EDA, *Electronic design automation*), HLS (*High-Level Synthesis*) indica la transformación sistemática de una especificación con un nivel de abstracción alto a una representación estructural en RTL que consigue el comportamiento dado (Figura 3.4).

Incrementar los niveles de abstracción *hardware* es esencial para evaluar a nivel de sistema decisiones sobre la arquitectura, tales como el particionado *hardware/software*, organización de memoria, gestión de consumo, etc [13].

La más reciente generación de metodologías de síntesis de alto nivel (HLS) está basada en C, y está principalmente orientada a las aplicaciones *datapath* [14]. Los lenguajes descripción *hardware* tales como Verilog y VHDL han permitido la amplia adopción de herramientas de simulación. Estos HDLs también han servido como entrada a las herramientas de síntesis lógica llevando a la definición de sus subconjuntos sintetizables [13].

Seguir con la investigación es vital, porque queda un largo camino hasta conseguir HLSs que automáticamente realicen diseños sin excesivas directrices por parte diseñador, y que generen resultados óptimos para las diferentes limitaciones de diseño y tecnologías.

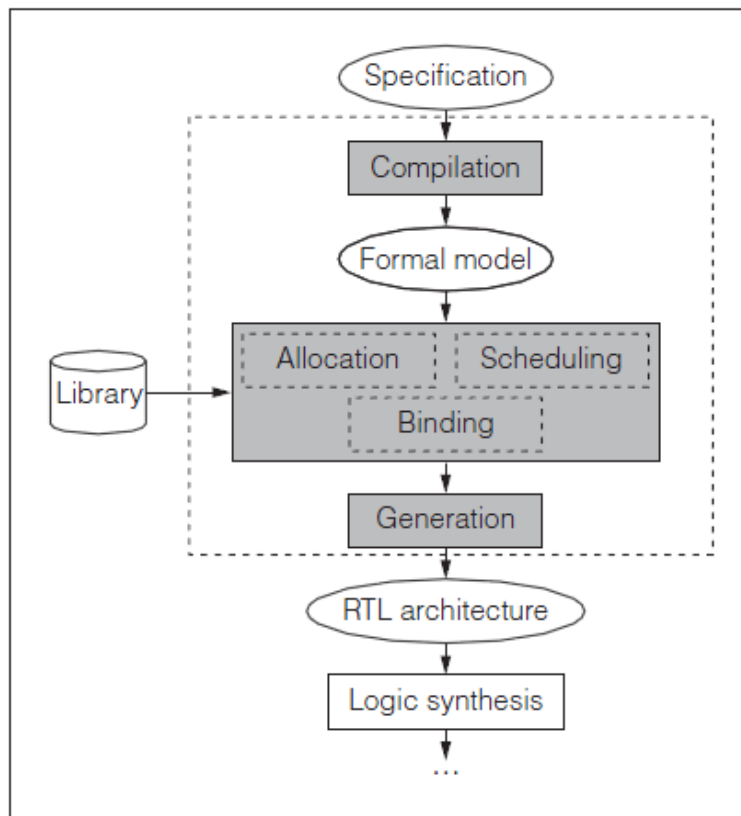


Figura 3.4- Pasos de HLS. (Fuente: [13])

4. Desarrollo e implementación

4.Desarrollo e Implementación

4.1. Diseño de módulos *hardware*

En este primer intento de acercamiento de dispositivos tipo FPGA, se ha diseñado una interfaz genérica para posibilitar la comunicación con el microprocesador, y dos módulos de ejemplo en VHDL para analizar su complejidad y posteriormente estudiar su alternativa de descripción mediante un lenguaje tipo C.

Se decidió diseñar un módulo generador de una señal PWM y un módulo generador de una señal para controlar tiras de LEDs direccionables. Se escogieron estos módulos para mostrar posibles aplicaciones de gran utilidad y versatilidad, además de su nivel medio de complejidad de cara a su futura descripción en C.

4.1.1.PWM

CONSIDERACIONES PREVIAS

La modulación por ancho de pulsos, también conocida como PWM de sus siglas en inglés (*Pulse-width Modulation*), es un tipo de señal empleada para enviar información o para controlar el suministro de energía que se envía a una carga.

Entre sus diversas aplicaciones hay que destacar su empleo en la emulación de señales analógicas en circuitos digitales y su utilidad para el control del suministro de energía a dispositivos electrónicos. Esta última es muy común, como por ejemplo en la regulación de la velocidad de giro de los motores eléctricos y servos utilizados en el campo de la Robótica.

El método analógico clásico de la generación de la señal PWM se basa en el uso de un comparador [9]. Una señal en forma de dientes de sierra triangulares es comparada con una tensión de referencia. La señal de salida está a nivel alto o nivel bajo dependiendo si la magnitud de la tensión de referencia es mayor o menor que la señal de dientes de sierra triangulares. El ciclo de trabajo puede ser cambiado modificando el nivel de la tensión de referencia.

Otra alternativa para generar señales PWM es usar un circuito digital mediante el empleo de un contador. En este método, un contador se incrementa periódicamente y dicho contador está ligado al reloj del circuito. El contador se resetea al final de cada periodo de la señal PWM. La salida PWM conmuta cuando el valor del contador excede la referencia. Este contador es el modelo discreto análogo a un diente de sierra. Una comparación lógica entre el valor del contador y el valor de la referencia digital es el sustituto del comparador analógico. El ciclo de trabajo puede ser solo alterado en pasos discretos, también como una

función de la resolución del contador. Un contador que tenga una resolución mayor puede proporcionar un comportamiento totalmente satisfactorio.

Es importante definir los parámetros fundamentales en los PWM:

- El **ciclo de trabajo** es el ancho relativo entre el tiempo que está a nivel alto (T_{on}) y el período de la señal (T). Su valor se suele expresar en porcentajes y puede variar de 0 a 100%. En la Figura 4.1 se muestra un pulso con un ciclo de trabajo del 50%.

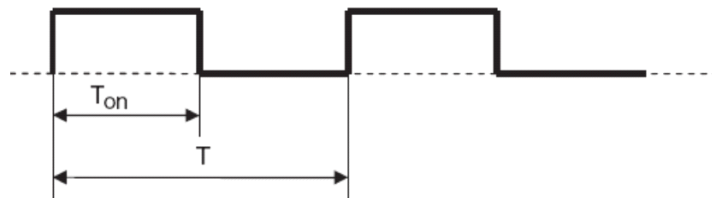


Figura 4.1-Parámetros de una señal PWM (Fuente: Autoría propia)

- La **resolución** es el número máximo de bits que resultan útiles para definir el ciclo de trabajo del PWM.
- La **frecuencia** del PWM será la frecuencia del reloj del circuito dividido entre el valor máximo del contador. La frecuencia del PWM está limitada por la máxima frecuencia del reloj de la FPGA.

La generación digital de un PWM es mejor que la analógica ya que tiene una mayor fiabilidad, menor complejidad y por la facilidad de generación de señales digitales.

DESCRIPCIÓN DEL DISEÑO

En este proyecto, la técnica digital propuesta implica incrementar un contador y comparar el valor de dicho contador con un valor precargado en un registro. El ciclo de trabajo puede ser cambiado modificando el valor del registro. La señal PWM estará a nivel alto mientras el valor de la cuenta sea menor que el del registro de comparación y cambiará a cero al alcanzarlo. El funcionamiento descrito se puede observar en la Figura 4.2.

En este diseño se ha utilizado un contador de 8 bits cuyo valor máximo de la cuenta es 255. Se utilizan dos registros de comparación para sacar dos señales PWM con distinto ciclo de trabajo si así se desea.

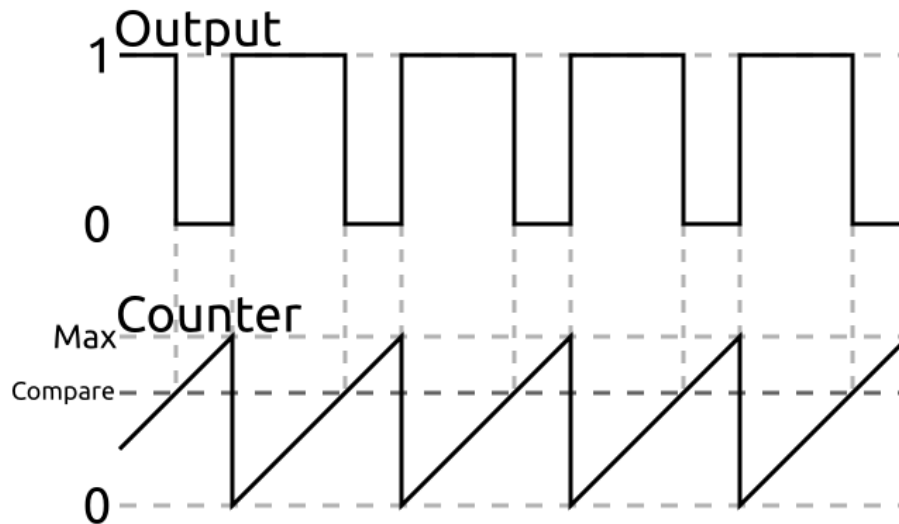


Figura 4.2- Generación de una señal PWM (Fuente: [5])

Con el reloj de la FPGA de la placa empleada en los ensayos, cuya frecuencia es de 50 MHz, se tiene una frecuencia máxima de la señal muy elevada. Para ampliar las posibles aplicaciones del PWM generado a aquellas que necesiten una frecuencia menor, en este módulo se incluye también la posibilidad de elegir la frecuencia de la señal PWM mediante un preescalado.

El preescalado se consigue utilizando otro contador dentro del cuerpo del mismo código que habilita la cuenta del contador del PWM cada cierto número de ciclos de reloj. Para la elección del preescalado se tiene un registro en el que se almacena el valor correspondiente a la división deseada.

ESTRUCTURA DEL MÓDULO

El módulo tiene siete entradas y tres salidas, además de tres registros sobre los que se puede leer y escribir. Estos registros son los dos registros de comparación “*compareA*” y “*compareB*”, uno para cada señal PWM, y el registro de selección del preescalado “*timersel*”.

Las entradas del módulo son:

- “clk”: señal de reloj.
- “reset”: activo por nivel bajo.
- “Din”: bus de datos de 8 bits procedente del microprocesador.
- “A”: bus de direcciones de 3 bits procedente del microprocesador.
- “DS”: *data strobe*, señal de control procedente del microprocesador; activa por nivel bajo.

- “CS”: *chip select*, señal de control procedente del microprocesador, activa por nivel alto.
- “RW”: *Read/Write*, señal de control procedente del microprocesador; ‘1’ para leer y ‘0’ para escribir.

Las salidas del módulo son:

- “Dout”: Bus de datos de 8 bits para el flujo de datos desde el módulo al microprocesador.
- “pwmA”: Señal PWM con un ciclo de trabajo fijado por el registro de comparación “*compareA*”.
- “pwmB”: Señal PWM con un ciclo de trabajo fijado por el registro de comparación “*compareB*”.

4.1.2. LEDS

CONSIDERACIONES PREVIAS

Como se mencionó anteriormente en los objetivos, se quiere crear bloques de control de dispositivos que puedan ser de interés a personas sin la experiencia o el conocimiento suficiente para programarlos. Un claro ejemplo es el del control de una tira de LEDs que puede ser de gran utilidad en proyectos de tipo artístico, educativo, etc.

En particular se ha diseñado este bloque para el control de los módulos de LEDs WS2812 y WS2812B. En la Figura 4.3 se muestran los esquemas de un LED de cada uno de los modelos distintos cuya principal diferencia es el número de pines/patillas. Estos módulos son LEDs RGB (Red-Green-Blue) cada uno con un circuito de control integrado (IC, Integrated Chip) [15].

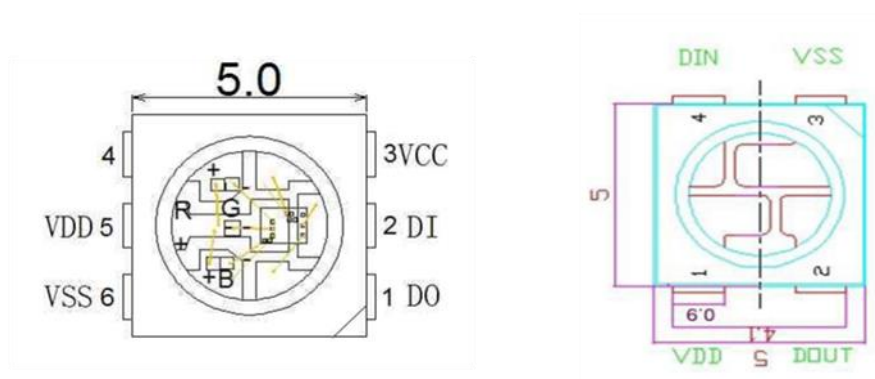


Figura 4.3- LEDs WS2812 (izquierda) y WS2812B (derecha) (Fuente: [15])

Su funcionamiento es realmente ingenioso; cada LED dispone de un integrado que almacena 3 bytes (24 bits), que corresponden con los 3 colores del RGB [15]. Cada pixel puede tener entonces 256 niveles en 3 colores, lo que supone un total de 16.777.216 posibles colores.

La secuencia de bits debe enviarse empezando por el más significativo hasta terminar con el menos significativo, mandando primero el byte correspondiente al color verde seguido del rojo y del azul como se puede observar en la Figura 4.4.

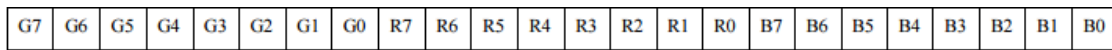


Figura 4.4- Composición de 24 bits de colores de un LED. (Fuente: [15])

La transmisión de datos se realiza mediante el protocolo de comunicación NRZ (non-return-to-zero) en el que tras el encendido y reset del pixel, el puerto de entrada recibe los datos del controlador [15].

Estos módulos son gobernados por una sola línea de datos, con los chips conectados en serie. El pin de salida de datos (DO) de uno, está conectado al pin de entrada de datos (DIN) del siguiente tal y como se muestra en la Figura 4.5.

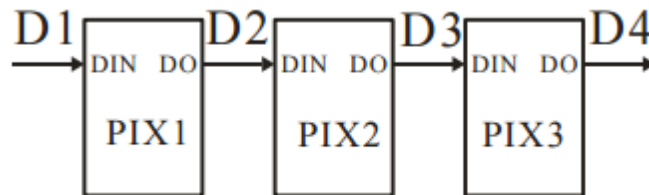


Figura 4.5- Método de comunicación en cascada entre módulos LED. (Fuente: [15])

Los datos correspondientes a los colores que se quieren mostrar se mandan bit a bit al primero de la cadena. Este "absorbe" los primeros 24 bits y pasa los sucesivos al siguiente LED como se puede ver en el esquema de la Figura 4.6. De esta manera se mandan 24 bits a cada módulo LED de la cadena. Finalmente, cuando se quiere que luzcan, se mantiene la señal a nivel bajo durante un tiempo suficientemente largo y los manda a los LEDs que utilizando una señal PWM mantienen el color hasta que se cargue y mande una nueva serie de colores [15].

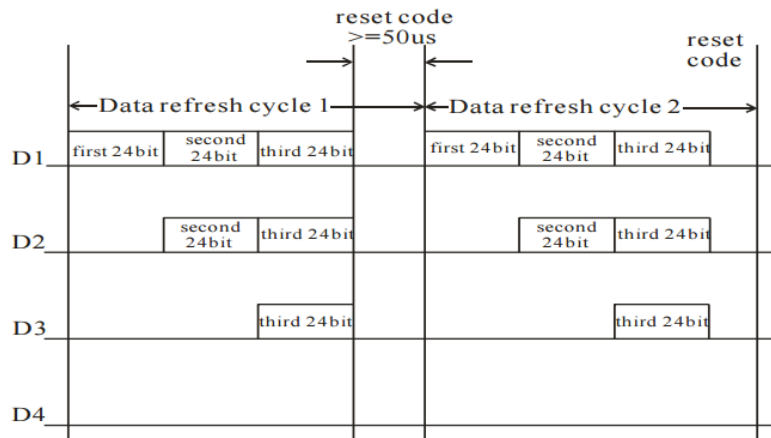


Figura 4.6- Método de transmisión de datos. (Fuente: [15])

Para que la tira de LEDs sea capaz de interpretar la señal que le llega, los bits son modulados de manera que la señal del controlador empieza a nivel bajo y se conmuta a nivel alto, señalando así el inicio de un bit. Como se puede observar de la Figura 4.7, según sea un '1' o un '0' el bit que se esté mandando, se mantiene a nivel alto un tiempo determinado, T1H y T0H respectivamente.

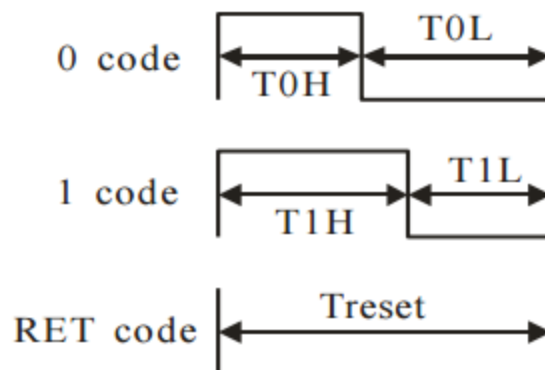


Figura 4.7- Codificación de bits empleada en el protocolo (Fuente: [15])

DESCRIPCIÓN DEL DISEÑO

Como se ha mencionado anteriormente, las tiras de LEDs WS2812 y WS2812B [15] trabajan a velocidades de transmisión de datos muy altas alrededor de los 800 Kbps (*Kilobits per second*), por lo que el envío de la secuencia de bits se hará mediante un módulo diseñado en una FPGA que trabaja a frecuencias mucho mayores que las de un microprocesador.

El funcionamiento básico consiste en registrar en la FPGA los datos provenientes del microprocesador que se desean enviar posteriormente a la tira de LEDs. Con la

información registrada, si se habilita la salida a los LEDs, se procede a generar la señal de salida para el control de estos.

Este módulo debe ser entonces capaz de comunicarse tanto con la tira de LEDs como con el microprocesador. En la comunicación con el microprocesador a través de la interfaz, se pueden recibir y mandar datos gracias a una serie de señales que son el bus de datos de entrada al módulo, el bus de datos de salida hacia el microprocesador, un bus de direcciones y las señales de control.

En el protocolo empleado siempre que se lea un dato de la memoria del módulo o que se escriba un dato en su memoria, se autoincrementa el valor del registro de direcciones para así leer o escribir, respectivamente, en posiciones sucesivas de la memoria.

Una vez almacenada la información necesaria, se procede a generar la señal de control de los módulos LED. Para ello se escribe en el registro de direcciones la posición desde la que se quiere tomar los datos que van a ser enviados. Se toma este dato y se empieza desde el bit más significativo (MSB) al menos significativo (LSB) modulando la señal según sea el bit un uno o un cero.

Para la generación correcta de la señal se han definido tres constantes:

- H1 es el número de ciclos de reloj que tiene que mantenerse a nivel alto la señal para codificar un uno.
- H0 es el número de ciclos de reloj que tiene que mantenerse a nivel alto la señal para codificar un cero.
- T es el periodo de un bit que será la suma del tiempo a nivel alto y el tiempo a nivel bajo.

Para el cálculo del valor de estas constantes se han tenido en cuenta los tiempos especificados en la hoja de especificaciones de los LED WS2812 y WS2812B [15] y se ha profundizado investigando casos prácticos donde se han utilizado estas tiras de LEDs. Conocida la frecuencia del reloj de la FPGA de la placa empleada en los ensayos (50 MHz) y la velocidad de transmisión de bits a los LEDs (800 Kbps), se muestran en la Tabla 4.1 los datos tomados [15] y los cálculos realizados.

	t (ns) especificaciones		t (ns) tomado	ciclos de reloj (clk)
	mínimo	máximo		
Bit cycle, T	1030	1250	1240	62
High time '1', H1	625	1250	700	35
High time '0', H0	62.5	500	400	20
Reset/Latch time, R	10800		11000	

Tabla 4.1- Especificaciones de tiempos de los LEDs WS2812 y WS2812B (Fuente: Autoría propia)

ESTRUCTURA DEL MÓDULO

A continuación se citarán y explicarán brevemente las entradas y salidas del módulo LEDs.

Las entradas del módulo son:

- “clk”: señal de reloj.
- “reset”: activo por nivel bajo.
- “DIN”: bus de datos de 8 bits procedente del microprocesador.
- “A”: bus de direcciones de 3 bits procedente del microprocesador.
- “DS”: *data strobe*, señal de control procedente del microprocesador; activa por nivel bajo.
- “CS”: *chip select*, señal de control procedente del microprocesador, activa por nivel alto.
- “RW”: *Read/Write*, señal de control procedente del microprocesador; ‘1’ para leer y ‘0’ para escribir.

Las salidas del módulo son:

- “DOUT”: Bus de datos de 8 bits para el flujo de datos desde el módulo al microprocesador.
- “DOUT_L”: Señal de control de los LEDs de un 1 bit.

Como se puede ver en la Figura 4.8, este módulo consta de un bloque de memoria y cuatro registros de 1 byte sobre los que se puede leer y escribir.

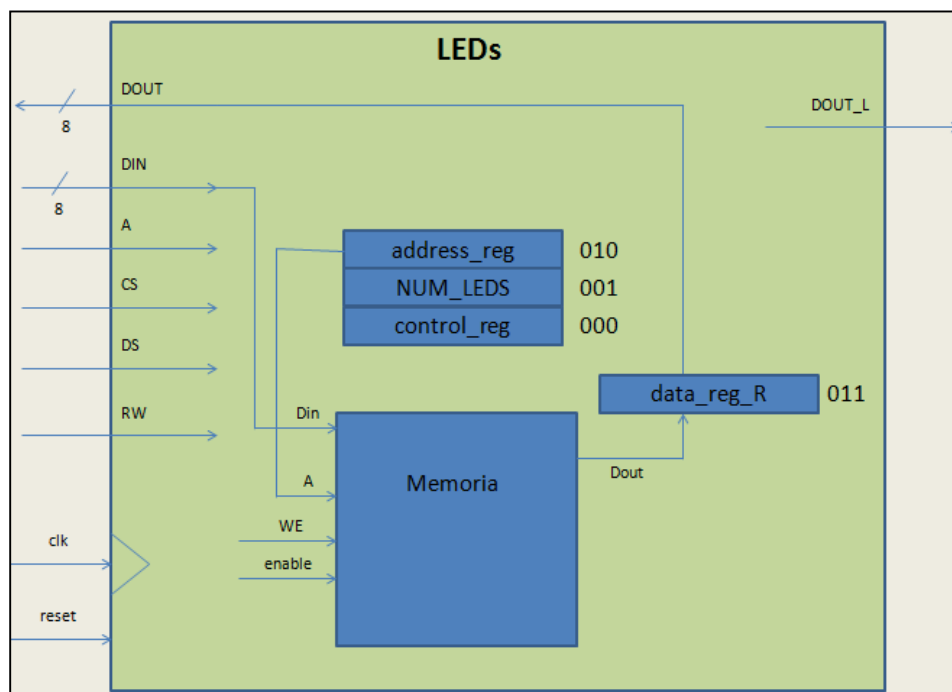


Figura 4.8 - Esquema módulo LEDs. (Fuente: Autoría propia)

El bloque de memoria es un vector bidimensional cuya capacidad es de $2^8 \times 8$ bits. Tiene como entradas: “clk” (señal de reloj), “reset” (activo por nivel bajo), “Din” (bus de datos de 8 bits conectado al bus de entrada de datos del módulo), “A” (bus de direcciones, para apuntar a una dirección dentro de la memoria), “enable” (habilitación del acceso a la memoria), y “WE” (*Write Enable*, habilita la escritura en memoria). Su única salida “Dout” (bus de datos de salida) tendrá siempre el dato contenido en la posición de memoria apuntada por el bus de direcciones.

Por otra parte, el primero de los registros es el de control (“control_reg”) sobre el que se escribe un ‘1’ en su bit menos significativo si se quiere habilitar la salida de la señal de control de los LEDs. El segundo es un registro en el que se guarda el número de LEDs (“NUM_LEDS”) que se van a manejar. El tercero es el registro de direcciones (“address_reg”) sobre el que se escribe la dirección de la memoria de datos a la que se quiere acceder. Y el último es el registro de lectura de datos (“data_reg_R”) que guarda el valor almacenado en la dirección de memoria apuntada por el registro de direcciones.

4.1.3. Interfaz

CONSIDERACIONES PREVIAS

La partición de diseño en *hardware* y *software*, implica dos códigos distintos y dos bloques que se tienen que comunicar entre sí.

Es necesario entonces planificar una interfaz *software/hardware* y diseñar un controlador *hardware* implementado en la FPGA para controlar esta interfaz [16]. Si el microprocesador se encontrase dentro de la FPGA, estaríamos ante un caso más simple ya que se tendrían accesibles los buses de datos y direcciones y la interfaz entre los dos dispositivos sería estándar.

Sin embargo, en la placa empleada para los ensayos, solamente están disponibles dos puertos para la comunicación entre el microprocesador y la FPGA. Debido a esto, se ha tenido que usar uno de los puertos como un bus bidireccional de datos y direcciones multiplexados, y el otro bus disponible para mandar las señales de control del microprocesador a la FPGA. Este diseño se ha realizado así, considerando que el microprocesador es siempre el “maestro” y la FPGA la “esclava”. En un futuro podría considerarse la posibilidad de que la FPGA pudiese interrumpir al microprocesador, con lo cual habría que revisar el diseño actual.

DESCRIPCIÓN DEL DISEÑO

Este módulo (Figura 4.9) se creó para hacer de interfaz entre el microprocesador y la FPGA, donde el microprocesador hace de maestro y la FPGA de esclava.

En él se define el protocolo usado para la escritura y la lectura entre el *software* del microprocesador y los bloques *hardware* diseñados en la FPGA. Existen cuatro posibles escenarios de comunicación entre microprocesador y FPGA que son:

- Escritura de una dirección (AWP)
- Escritura de un dato (DWP)
- Lectura de una dirección (ARP)
- Lectura de un dato (DRP)

La interfaz interpreta las señales de control procedentes del microprocesador (DS, AS, RW, MODO) y decide en qué escenario está.

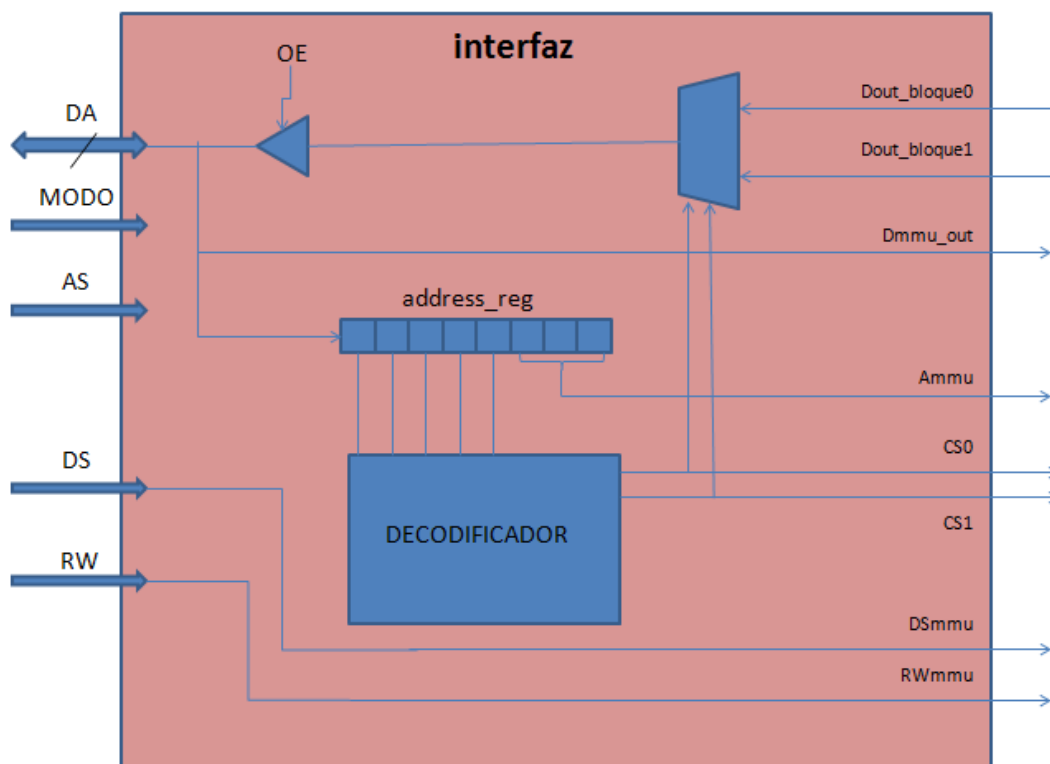


Figura 4.9- Esquema módulo interfaz genérica. (Fuente: Autoría propia)

Para distinguir los escenarios de lectura o escritura existe la señal de *Read/Write* en la que el microprocesador pondrá un uno en caso de querer leer un dato proveniente de la FPGA o un cero en caso de querer mandar un dato a esta misma.

Al ser un bus donde datos y direcciones están multiplexados, es necesario apoyarse en las señales de *data strobe* y *address strobe* para diferenciar los dos posibles casos. Es decir, si se quiere escribir o leer un dato será necesario bajar la señal de *data strobe*; y por el

contrario, si lo que se quiere escribir o leer es una dirección se bajará la señal de *address strobe*.

El bus bidireccional se gestiona mediante una puerta triestado que permitirá la salida de datos desde la FPGA al microprocesador cuando se habilite la señal de control de la puerta (OE), que será en los casos de lectura. En los casos de escritura del microprocesador sobre la FPGA, se pasará el dato o la dirección a través de este bus bidireccional.

Al coexistir varios módulos o bloques *hardware* dentro de la FPGA, hay que especificar sobre qué bloque se quiere escribir o leer. Para esto se escribe en el registro de direcciones de la interfaz (“*address_reg*”) donde los 5 bits más significativos se decodifican y generan las señales de chip select (de salida del bloque de la interfaz) que se utilizan para la habilitación del acceso a los registros de cada uno de los módulos de la FPGA.

ESTRUCTURA DEL MÓDULO

Como se muestra en la Figura 4.10 el módulo tiene nueve entradas y seis salidas.

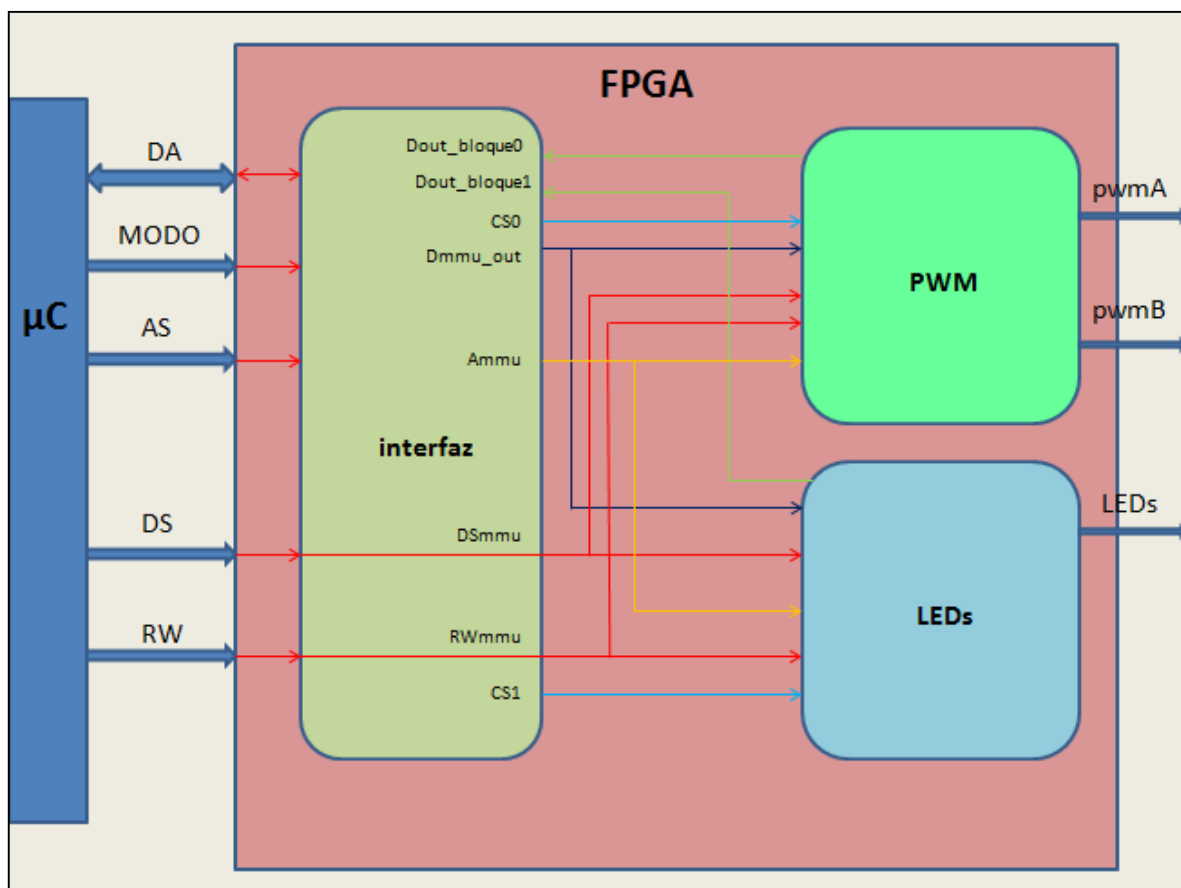


Figura 4.10- Esquema del módulo interfaz genérica. (Fuente: Autoría propia)

Las entradas del módulo se pueden dividir entre las de comunicación con el microprocesador y las internas a los bloques *hardware* de la FPGA:

➤ Con el microprocesador

- “DA”: bus bidireccional multiplexado para datos y direcciones entre la FPGA y el microprocesador.
- “DS”: *data strobe*, señal de control procedente del microprocesador; es activa por nivel bajo e indica que hay una dirección en el bus de datos y direcciones.
- “AS”: *address strobe*, señal de control procedente del microprocesador; es activa por nivel bajo e indica que hay una dirección en el bus de datos y direcciones.
- “RW”: *Read/Write*, señal de control procedente del microprocesador; se emplea un ‘1’ para leer y un ‘0’ para escribir.
- “MODO”: señal de control procedente del microprocesador para decidir si se está en modo normal o modo ráfaga.

➤ Interno a la FPGA

- “clk”: señal de reloj.
- “reset”: activo por nivel bajo.
- “Dout_bloque0”: bus de datos procedente del módulo PWM de la FPGA.
- “Dout_bloque1”: bus de datos procedente del módulo LEDS de la FPGA.

Las salidas del módulo son:

- “Dmmu_out”: bus de datos de 1 byte hacia los módulos *hardware*.
- “Ammu”: bus de direcciones de 3 bits hacia los módulos *hardware*.
- “DSmmu”: *data strobe*, señal de control que manda pulso a nivel bajo.
- “RW”: *Read/Write*, señal de control hacia los módulos *hardware*; ‘1’ para leer y ‘0’ para escribir.
- “CS0”: *chip select 0*, señal de habilitación del acceso a los registros del módulo PWM.
- “CS1”: *chip select 1*, señal de habilitación del acceso a los registros del módulo LEDS.

4.2. Simulación e implementación

4.2.1. Simulaciones

La plataforma empleada para el diseño de los módulos y la escritura de su código ha sido el Xilinx ISE Design Suite. Este entorno facilita el diseño de las FPGAs desde principio hasta fin. Es una herramienta usada para el diseño de FPGAs, que ofrece la posibilidad de síntesis de HDLs, realizar simulaciones, implementación del diseño y programación de los dispositivos.

Una vez descritos los módulos *hardware* en VHDL y conseguido el diseño global (PWM, LEDs e interfaz), se procedió a la realización de simulaciones para comprobar su correcto funcionamiento.

Para facilitar la interpretación de los resultados de las simulaciones, se han separado las simulaciones del bloque PWM y las del módulo de los LEDs.

PWM

En la Figura 4.11 se muestra el proceso de configuración de los parámetros del PWM donde los valores dados a los registros son: “*compareA*”=32, “*compareB*”=16 y “*timersel*”=8.

En la configuración se escribe primero en el registro de direcciones la dirección del registro “*compareA*” y después se manda el dato deseado. Se repite esto mismo para escribir en los registros “*compareB*” y “*timersel*”.

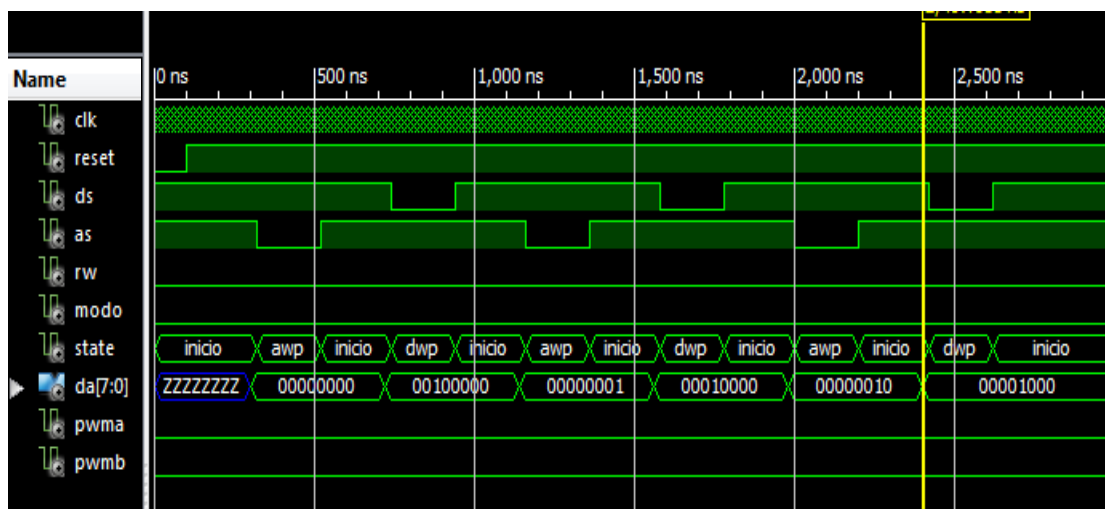


Figura 4.11- Configuración PWM. (Fuente: Autoría propia)

Con la configuración descrita, se muestran las salidas PWM obtenidas en la Figura 4.12.

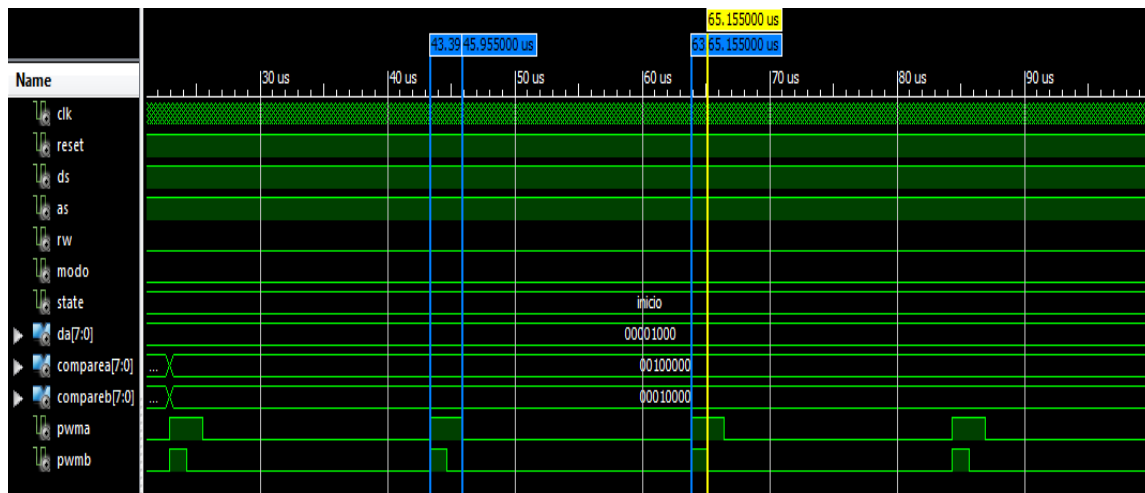


Figura 4.12- Salidas PWM. (Fuente: Autoría propia)

Cambiando los parámetros del PWM sobre los registros a los valores “compareA”=64, “compareB”=8 y “timersel”=16, se pueden observar diferencias entre la Figura 4.12 y la Figura 4.13. Ahora el ciclo de trabajo de la señal “pwma” es mucho mayor ya que se ha aumentado de un 12.5% a un 25%. Y en el caso del “pwmb”, es justamente lo contrario ya que se ha reducido su ciclo de trabajo a la mitad.

También es interesante analizar la variación del período de los PWMs. Esto es debido al cambio en el registro “timersel”, que rige el preescalado. Ahora se tiene el doble de preescalado, es decir, si se observa la escala de tiempos, se verá que el período de las señales PWM es el doble.

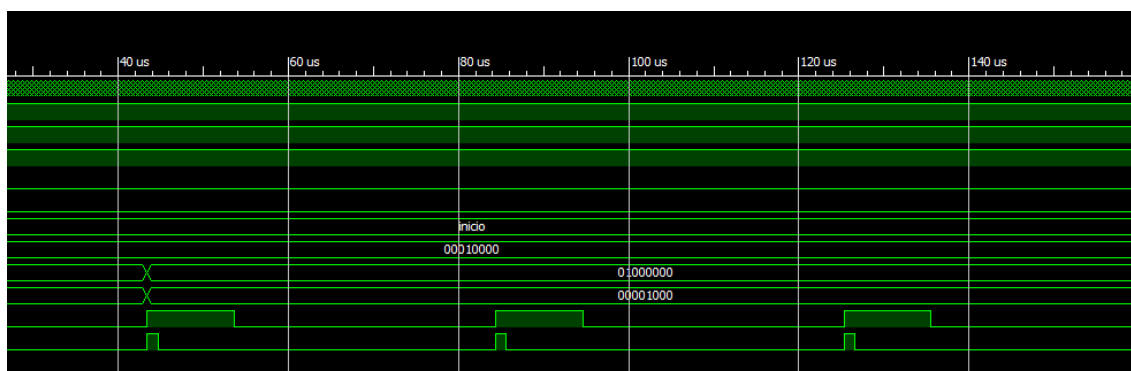


Figura 4.13- Salida PWM con cambio de parámetros. (Fuente: Autoría propia)

LEDs

Observando la Figura 4.14, podemos distinguir la primera fase de configuración donde se guardan en la memoria los datos que quieren ser enviados a los LEDs, y una segunda fase donde se manda la secuencia de bits a la tira de LEDs (señal inferior).

En la Figura 4.15, se ofrece una imagen ampliada de la salida de LEDs para distinguir la codificación de ‘1’s y ‘0’s.

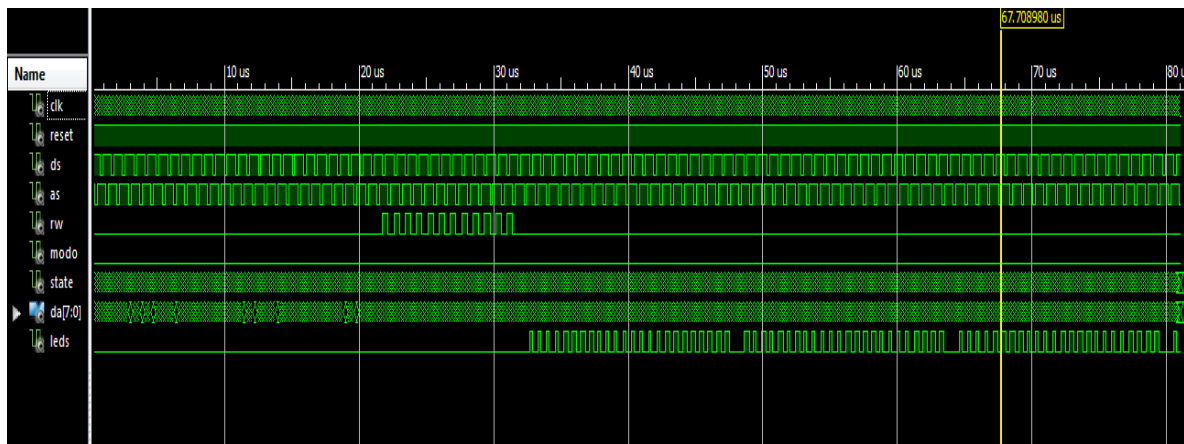


Figura 4.14- Salida LEDs. (Fuente: Autoría propia)

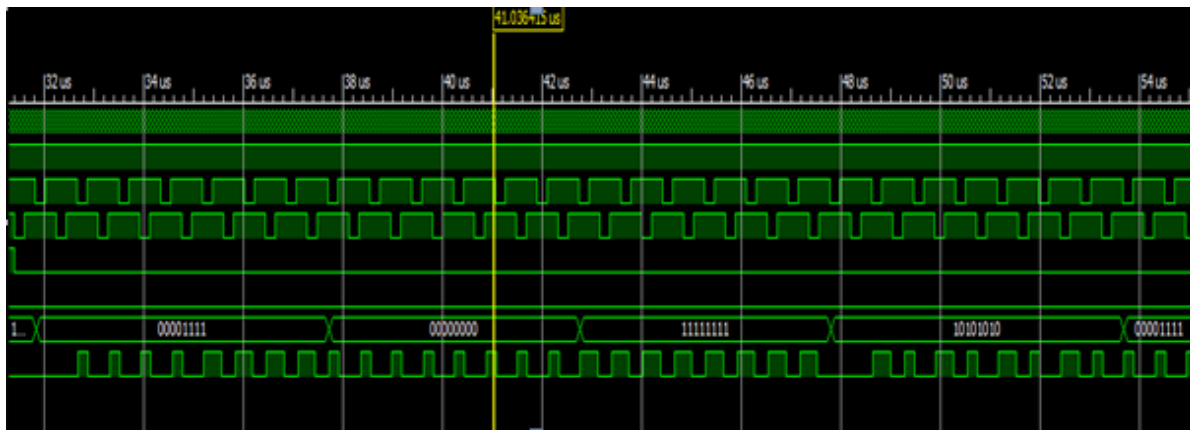


Figura 4.15- Salida a la tira de LEDs ampliada. (Fuente: Autoría propia)

4.2.2. Implementación

Como ya se mencionó en apartados anteriores, para los ensayos realizados en este proyecto, se ha utilizado una placa diseñada en la División de Ingeniería Electrónica de la Escuela Técnica Superior de Ingenieros Industriales.

Esta placa incluye un microcontrolador ATmega644PA de la familia AVR, que cuenta con un reloj de 16 MHz, y una FPGA XC3S200 de Xilinx cuyo reloj es de 50 MHz.

La implementación del diseño en la placa se puede dividir en dos partes (Figura 4.16). La primera consiste en la programación de la FPGA desde el entorno de desarrollo de Xilinx mencionado en el apartado anterior. La segunda consiste en la creación del programa ejemplo en C y la programación del microprocesador usando el Atmel Studio.

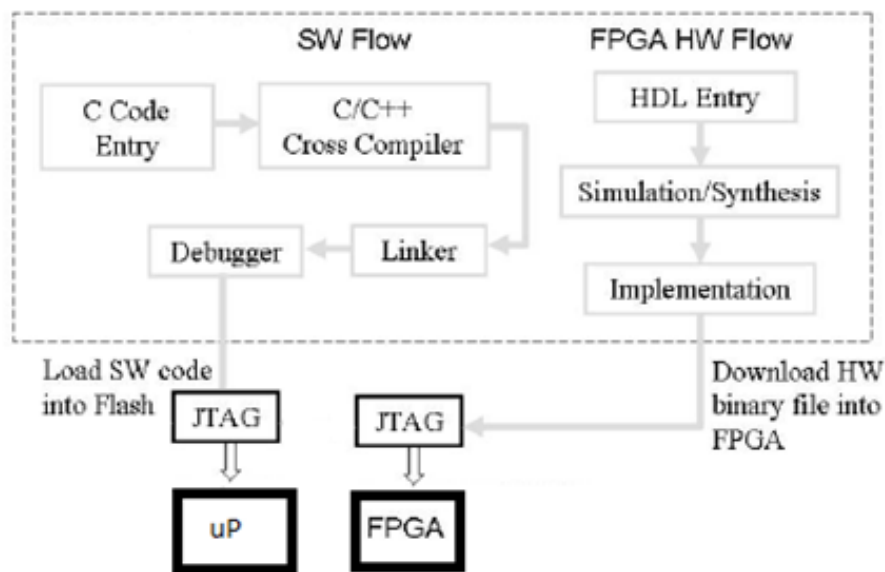


Figura 4.16- Flujo de diseño de microprocesador y FPGA. (Fuente: [16])

En las pruebas realizadas, el microprocesador enviaba los datos deseados a los registros necesarios para fijar los parámetros del módulo PWM. Para el módulo de los LEDs, el microprocesador mandaba a la FPGA los datos de los colores que se querían mostrar por la tira de LEDs.

Una vez cargados los programas en sus respectivos dispositivos, se observaron las señales de salida en el osciloscopio. Los resultados eran correctos en la generación de las señales PWM para cualquier ciclo de trabajo elegido. Sin embargo, la señal enviada a la tira de los LEDs presentaba errores en cuanto a los bits que codificaba.

Después de un detallado análisis, las conclusiones a las que se llegaron mostraban que existía un problema de sincronismo entre el microprocesador y la FPGA. Debido a este problema, los procesos de escritura y lectura de datos entre los dos dispositivos

programables de la placa no eran siempre correctos y producían errores en el funcionamiento. Estos errores se tendrán que estudiar más adelante haciendo un análisis en más detalle.

5. Propuesta de descripción *hardware* mediante C

5. Propuesta de descripción *hardware* mediante C

5.1. Subconjunto de C y método de traducción a VHDL

En este proyecto se ha querido aportar una propuesta preliminar de un subconjunto del lenguaje C que permita describir de forma sencilla diseños *hardware*. Para ello se ha planteado una estructura “tipo”, o esqueleto (Figura 5.1), con ciertas restricciones en cuanto al tipo de datos y sentencias del lenguaje que se podrán utilizar.

Los tipos de datos que se van a emplear en este subconjunto serán solamente tipo entero y tipo *char*. Se ha tomado esta decisión apoyada en dos razones. Por un lado, se pretende facilitar la descripción por parte del usuario, que no tendrá que incorporar nuevos tipos a su forma de trabajar. Por otro, se quiere facilitar el proceso de traducción, o preprocesado, desde C a VHDL, es decir la síntesis de alto nivel. Para ello se ha tenido en cuenta que el optimizador eliminaría aquellos excesos innecesarios, como por ejemplo en el caso de emplear un tipo entero para un *std_logic* de VHDL.

Por otro lado, y para esta primera propuesta, también se ha eliminado la posibilidad de utilizar sentencias tipo bucle (ya sean *while*, *do* o *for*) ya que, aun teniendo límites estáticos, lo que implica que serían teóricamente sintetizables, suponen un mayor problema para su preprocesado y síntesis. En cualquier caso, y como se ha comentado, esto es una propuesta preliminar, que se irá mejorando en posteriores trabajos.

Los módulos *hardware* que se diseñen en C se incorporarán como bloques con una interfaz común, tal y como se ha mostrado en el capítulo 4. Esta interfaz permite que el microprocesador acceda a los registros de configuración del módulo, si es que dispone de ellos. Por su parte, es muy posible que el módulo disponga por otro lado de registros internos para desarrollar su funcionalidad (excepto, claro está, en los casos que el diseño responda a una lógica combinacional, que será menos frecuente). Es necesario por tanto diferenciar entre los registros accesibles por el microprocesador, que llamaremos *registros globales*, y los registros internos al módulo, que llamaremos *registros locales*, siguiendo una nomenclatura similar a la utilizada en el software con las variables.

El planteamiento inicial es separar la declaración de *registros globales* y *registros locales*. Los primeros se mapearán en C como variables de tipo global, mientras que los *registros locales* al módulo, que no deberían ser accesibles desde el micro, parecería lógico que se declarasen como variables locales. Sin embargo, y desde un punto de vista *hardware*, estos registros deben mantener el valor entre ciclos de reloj del sistema (o si pensamos en la descripción VHDL, entre ejecuciones del proceso donde se describen), mientras que las variables locales no mantienen el valor entre llamadas a la función en la que están declaradas. Si bien esto no plantea problemas desde el punto de vista de la síntesis, sí que puede resultar confuso para un usuario formado en programación *software*. Por esta razón

(y para permitir además la *emulación software* de la descripción, de la que se hablará más adelante) se declararán estas variables como globales, y si acaso se calificarán como estáticas (*static*, en C, lo que las hace sólo visibles dentro del fichero en el que están declaradas). Para no confundir al usuario y, por otra parte, guiar al preprocesador en el proceso de síntesis, se utilizarán sentencias del preprocesador de C tipo *#pragma*, como se verá posteriormente.

Además se propone definir dos funciones por módulo *hardware*. Por una parte, una función de inicialización (llamada *SetupHardware*) utilizada para la asignación de los pines de las entradas y salidas y otras inicializaciones que se vean necesarias. Por otra parte, se tendrá la función principal (llamada *HardwareLoop*) donde se incluirá la descripción de la funcionalidad del módulo *hardware*. Esta función sería equivalente a lo que ocurre en *hardware* en cada ciclo de reloj, y se mapeará posteriormente en un proceso VHDL.

Desde el punto de vista del usuario, este puede interpretar la funcionalidad como un programa que corre en paralelo con su código C principal, pero ejecutándose a una frecuencia de reloj muchísimo mayor a la del reloj del microprocesador. Se pretende, sin embargo, que el usuario vaya modificando su manera de pensar y concebir el hardware, desde esa interpretación inicial paralela al *software*, hasta una interpretación más ajustada a la que se necesita para diseñar módulos *hardware*. Para esta transformación en la forma de pensar es imprescindible un conjunto de ejemplos que irán guiando al usuario en el proceso de aprendizaje. Aunque en este trabajo no se ha afrontado esta parte, se considera un elemento fundamental de la metodología que se está desarrollando.

Finalmente, se ha suprimido la posibilidad de incluir componentes tal y como se hace en VHDL (elementos descritos de forma independiente que se instancian en la descripción del módulo en cuestión). La alternativa propuesta a este problema es que la funcionalidad que debería ir en esos componentes se describa mediante el empleo de distintas funciones a las que se llame desde la función principal del *hardware*, siendo el preprocesador de síntesis el que las convierte a su vez en componentes, o más probablemente declarándose como funciones o procedimientos a incluir en el proceso VHDL que implementa la funcionalidad (este aspecto no está aún decidido, ya que requiere más experimentación antes de adoptar la mejor solución).

A continuación se expone un ejemplo de la estructura explicada a lo largo de este apartado en la Figura 5.1.

```
#pragma HARDWARE

#pragma HARDWARE_GLOBALS
uint8_t MAX;

#pragma HARDWARE_LOCALS
uint8_t count=0;

void SetupHardware(){

    assignOutput(led,13);

}

void HardwareLoop(){

    if (count == MAX) {
        count=0;
    }
    else {
        count=count+1;
    }

    if (count == MAX) led = 0;
    else led = 1;
}
```

Figura 5.1- Ejemplo de estructura propuesta. (Fuente: Autoría propia)

Para facilitar la traducción de C a VHDL, se emplearán etiquetas que el preprocesador debe reconocer. Al encontrarse la etiqueta `#pragma HARDWARE` debe crear un componente en VHDL cuyo nombre sería el especificado en la pestaña de Arduino (Figura 5.2). Las entradas y salidas de este componente serán las preestablecidas para la comunicación con el microprocesador (bus de datos, de direcciones y señales de control), el reloj y el reset, y aquellas variables declaradas a las que se les asignen pines de entrada/salida en el `SetupHardware`.

Con la etiqueta `#pragma HARDWARE_GLOBALS`, las variables se traducirían a registros en VHDL accesibles por el microprocesador (Figura 5.3). En el código VHDL serían necesarias varias declaraciones. La primera sería la declaración de señales internas en la arquitectura e inicializarlas en el reset. La segunda sería declarar en el proceso variables asociadas a las que se les asigna el valor de la señal para así operar con ellas dentro del proceso siguiendo el esquema secuencial equivalente a la descripción en C que ha realizado

el usuario. Finalmente el valor de las variables se asignaría a sus señales correspondientes al final del proceso.

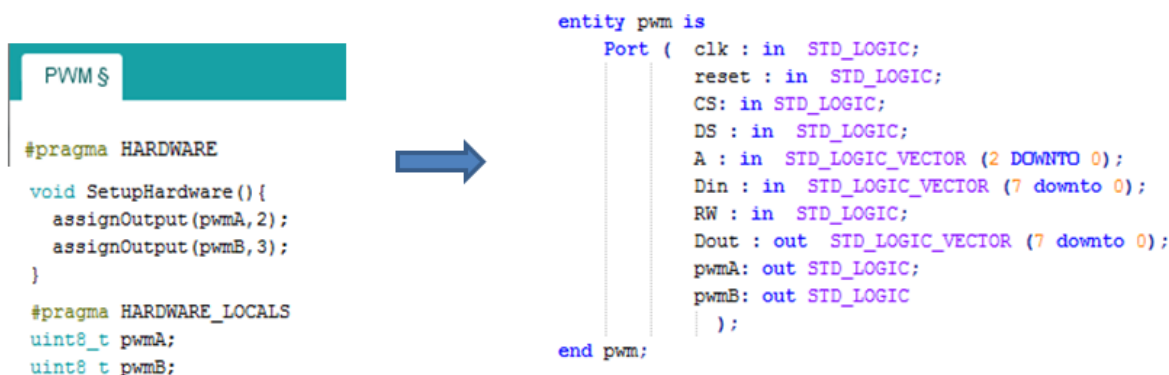


Figura 5.2- Ejemplo de creación del componente en VHDL desde C. (Fuente: Autoría propia)

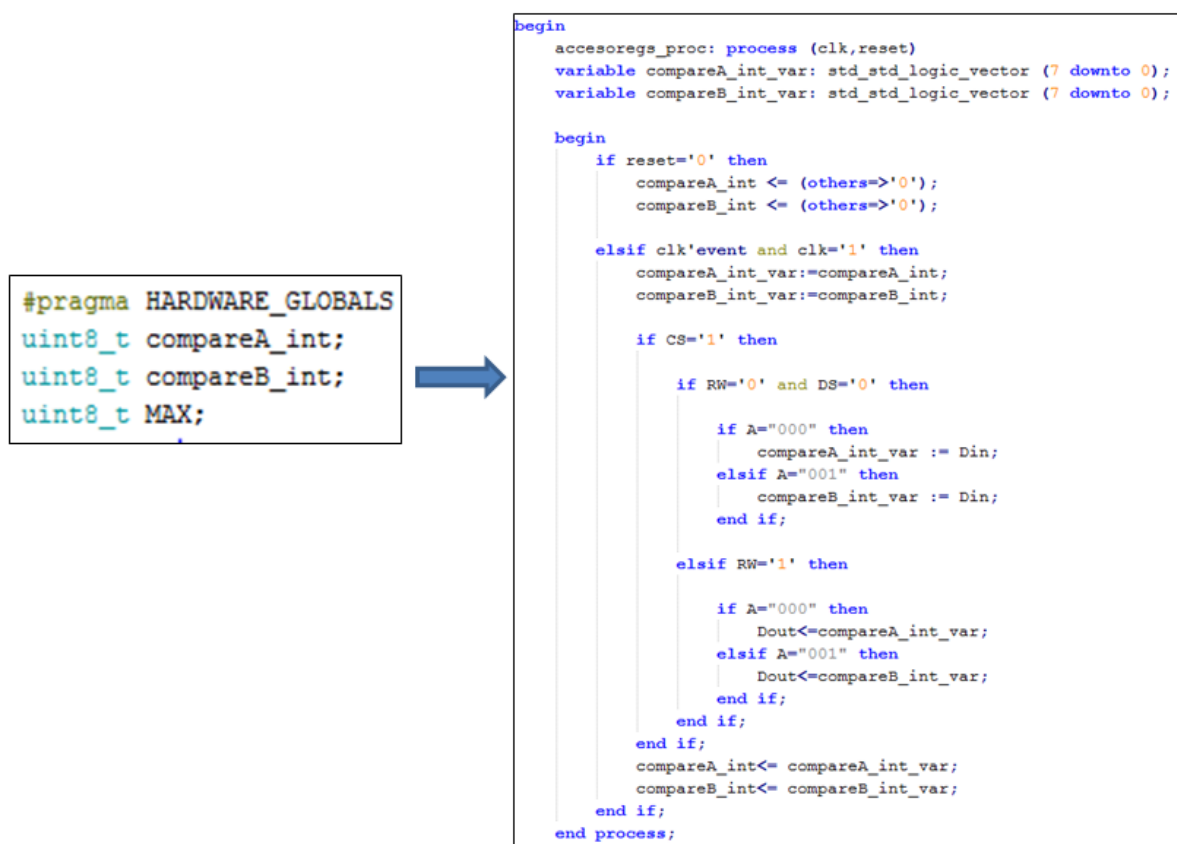


Figura 5.3- Ejemplo de declaración de las variables globales en VHDL desde C. (Fuente: Autoría propia)

De manera parecida, pero sin necesidad de declarar señales internas, la etiqueta `#pragma HARDWARE_LOCALS` indicaría la declaración de variables locales dentro del proceso del

hardware. A estas señales se las inicializaría al valor especificado en la descripción en C en caso de reset (Figura 5.4), y se operaría con ellas dentro del proceso de la manera especificada en el *HardwareLoop* (cuya traducción se muestra en la Figura 5.5).

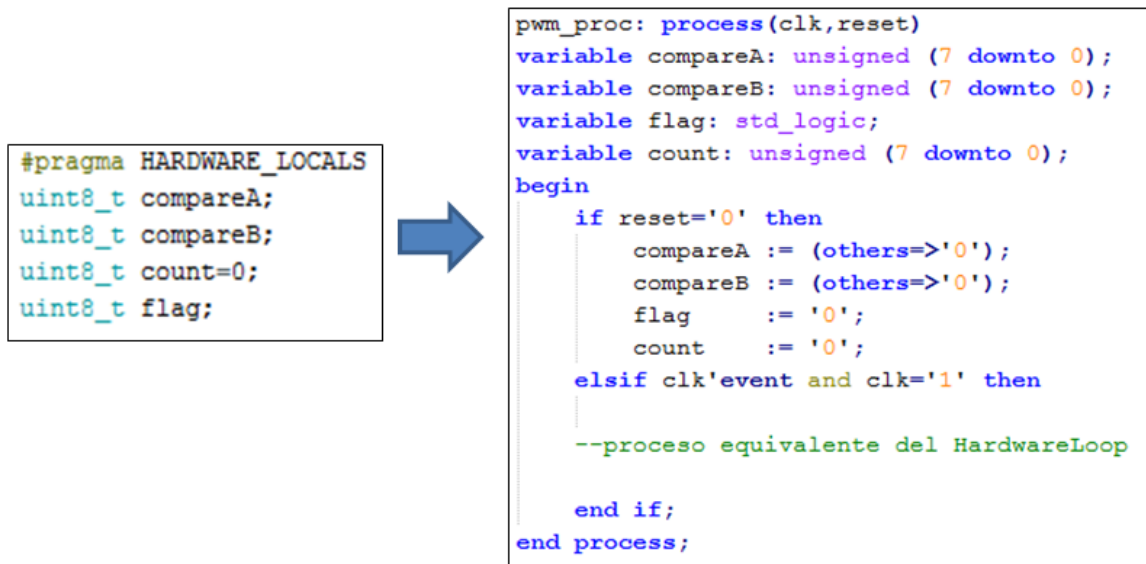


Figura 5.4- Ejemplo de declaración de las variables locales en VHDL desde C. (Fuente: Autoría propia)

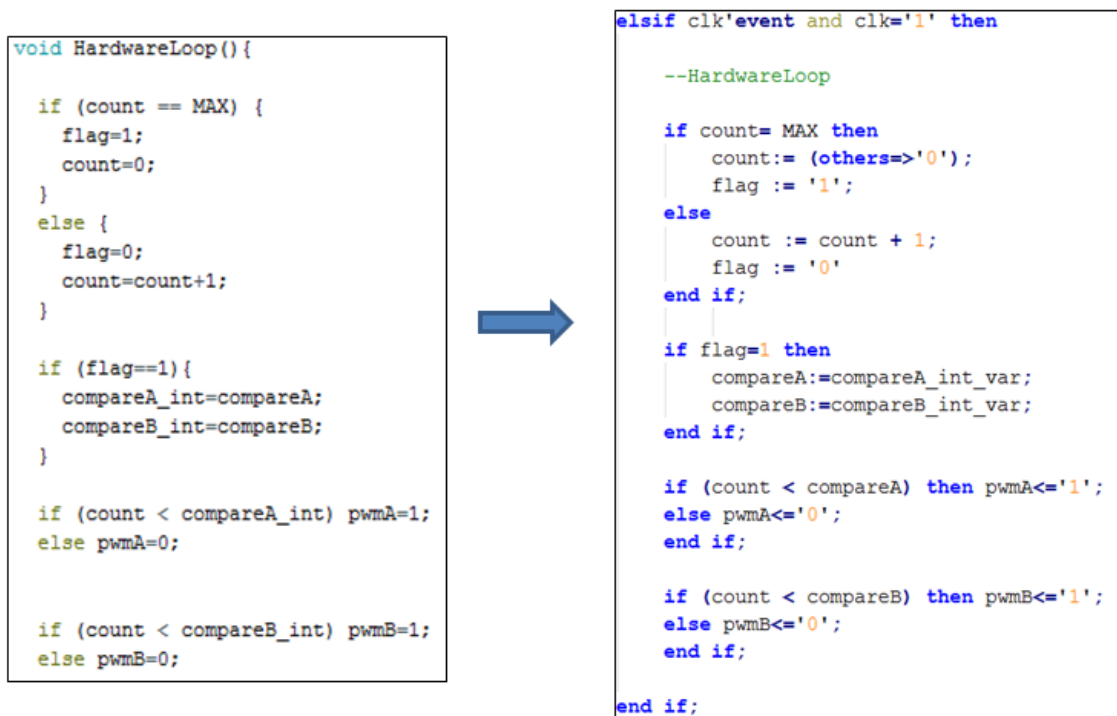


Figura 5.5- Ejemplo de traducción del HardwareLoop. (Fuente: Autoría propia)

5.2. Ejemplos de aplicación

Una vez descritos los módulos en VHDL y comprobado su correcto funcionamiento, se procedió a realizar un análisis sobre la descripción *hardware*. Este análisis se hizo con especial atención a la estructura y a la sintaxis de la descripción, pensando a su vez en una posible traducción de C a HDLs.

Después de estudiar los módulos descritos en *hardware*, el siguiente paso fue su descripción en C mediante el entorno de desarrollo de Arduino, intentando no producir muchas modificaciones respecto al código ya escrito en VHDL.

Los resultados fueron óptimos ya que se consiguió de manera rápida una descripción sencilla y correcta. Para comprobar el correcto funcionamiento, al menos desde el punto de vista funcional, se ejecutó la función *HardwareLoop* cada 1 ms a través de una interrupción periódica. De esta manera se pretendía comprobar que las señales evolucionaban de forma equivalente en la descripción en C y en la descripción VHDL, aunque a nivel temporal fuese distinto. Al ser el periodo de la interrupción muy largo con respecto al tiempo de ejecución de la función por parte del microprocesador, éste puede considerarse despreciable, y globalmente la evolución de las señales ocurre como si cambiasen de estado simultáneamente, sincronizadas con un hipotético reloj de periodo 1 ms. Se asignaron por tanto las señales PWM y la señal generada para controlar la tira de LEDs a pines digitales de la placa de Arduino y se analizaron en el osciloscopio.

Como se puede observar en las figuras Figura 5.6, Figura 5.7 y Figura 5.8, las señales obtenidas eran las deseadas y era posible cambiarlas en el programa en C mediante una simple modificación del valor de algunas variables (precisamente la *variables globales* que se corresponden con los *registros globales* de configuración de cada módulo).

La figuras Figura 5.6 y Figura 5.7 muestran las señales generadas por el módulo PWM con diferentes ciclos de trabajo, mientras que la Figura 5.8 muestra la secuencia de bits para controlar una tira de LEDs una vez que el microprocesador ha configurado adecuadamente la memoria del módulo y ha disparado el proceso de generación

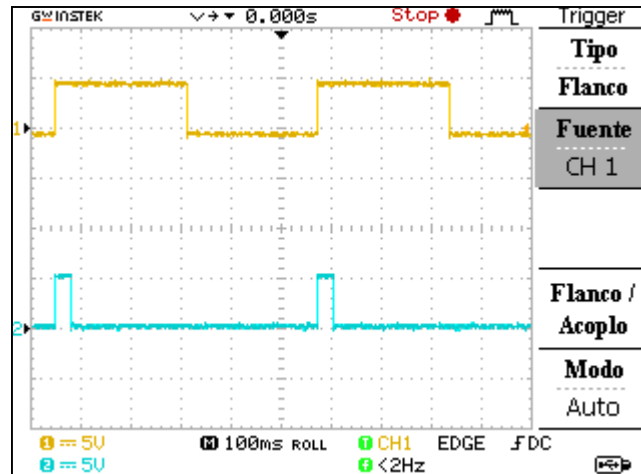


Figura 5.6- PWM con duty cycle=128 (arriba) y duty cycle=8 (abajo). (Fuente: Autoría propia)

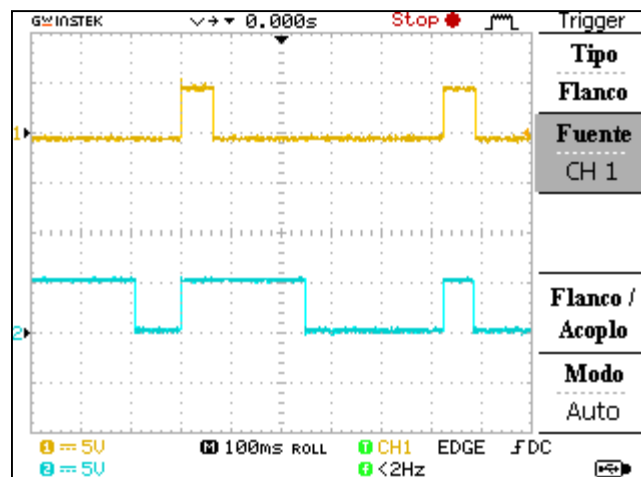


Figura 5.7- PWM con duty cycle=16 (arriba) y PWM con actualización de su duty cycle en cada período (abajo). (Fuente: Autoría propia)

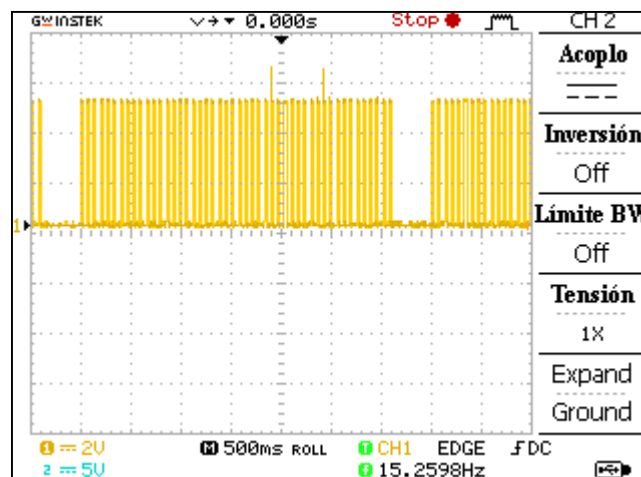


Figura 5.8- Secuencia de bits modulados para controlar tiras de LEDs. (Fuente: Autoría propia)

5.3. Emulación software

Al describir el hardware por parte del usuario, este no dispone de ningún procedimiento para verificar que la funcionalidad escrita es la correcta. Al contrario que las descripciones en C, que pueden ser depuradas con procedimientos de bajo coste (como trazas pasadas en *prints*), el usuario no puede depurar adecuadamente el hardware si no dispone del equipo necesario (osciloscopios, analizadores digitales de señal, etc.) o de simuladores. Por ello se ha querido tratar la posibilidad de “emular” el comportamiento del *hardware* mediante el propio programa que el usuario desarrollado.

La idea de la emulación es que el usuario pueda ejecutar el código C que describe la funcionalidad del hardware como cualquier otro programa y que pueda ver los resultados desde el punto de vista funcional. Desde un punto de vista formal, podemos pensar que al finalizar la ejecución software de la función *HardwareLoop*, las variables que luego se convertirán en señales *hardware* están actualizadas como lo estarían sus equivalentes *hardware* tras el flanco de reloj que las actualiza.

Para emular adecuadamente cada módulo, el preprocesador debería incluir una llamada a la función *HardwareLoop* al final del bucle principal del programa del usuario (el final de la función *loop*, si se trabaja bajo el entorno de Arduino). Cada una de las llamadas a la función *HardwareLoop* sería equivalente a un ciclo de reloj del *hardware* que implementará la funcionalidad descrita. Como están identificadas las señales que son entrada/salida de la FPGA (ya que se han descrito en la función *HardwareSetup*), se enviaría a través del puerto serie el valor de éstas señales junto con el número de ciclo de reloj al que corresponde. Posteriormente, una herramienta incluida en el entorno de desarrollo mostraría gráficamente la evolución de las señales deseadas como si de una simulación se tratase, ajustando los datos de tiempo a las frecuencias de reloj a la que trabaje la FPGA.

Hay que resaltar que la emulación es una propuesta a incluir en la futura plataforma, pero cuya viabilidad se ha mostrado a través de las emulaciones mediante interrupciones periódicas expuestas en el anterior apartado.

6. Conclusiones, Impactos y Líneas Futuras

6. Conclusiones y líneas futuras

6.1. Conclusiones

El principal objetivo de este trabajo era el estudio viabilidad del diseño *hardware* mediante una descripción en lenguajes de programación como C o similares para facilitar el uso de estas tecnologías a usuarios no expertos.

Para conseguir alcanzar esta meta, se dividió el proyecto en una serie de objetivos más concretos que se exponen a continuación a modo de recordatorio:

1. Investigación en el ámbito “Do It Yourself” de los proyectos ya existentes en el campo de introducción a la electrónica.
2. Diseño de dos módulos tipo, descritos en el lenguaje de descripción *hardware* VHDL, para ser implementados en una FPGA.
3. Desarrollo de una interfaz genérica entre microprocesador y FPGA integrados en una misma placa.
4. Aplicación práctica para comprobar el correcto funcionamiento del sistema en la placa diseñada en el DIE que se mencionó al comienzo de este capítulo.
5. Planteamiento de una posible y futura traducción de VHDL a C para facilitar el uso de las FPGAs a usuarios sin experiencia en programación de *hardware*.

El **primer objetivo** se ha cumplido, realizando una investigación en el ámbito “Do It Yourself” para adquirir conocimientos sobre los proyectos ya existentes y exitosos en este campo. Se ha hecho un análisis de las características de estos proyectos y se ha destacado Arduino como referencia en el desarrollo futuro de la plataforma que se pretende crear.

Como se ha podido comprobar en el capítulo 4 (desarrollo e implementación), el **segundo objetivo** se ha logrado realizando un diseño *hardware* de los módulos PWM y LEDs en VHDL. Se puede ver la descripción del diseño de los bloques y además comprobar su correcto funcionamiento en las simulaciones mostradas. La conclusión a la que se ha llegado tras el diseño de estos módulos es que, aun describiendo funcionamientos muy simples, puede complicarse la tarea para el desarrollador.

En cuanto al **tercer objetivo**, se consiguió describir una interfaz genérica en VHDL para lograr una comunicación adecuada entre el microprocesador y la FPGA integrados en la misma placa. De esta manera, se permite la escritura y lectura de la FPGA mediante las órdenes mandadas por parte del microprocesador.

Para llevar a cabo la aplicación práctica señalada en el **cuarto objetivo**, se implementaron los módulos PWM y LEDs y la interfaz en la FPGA, y se desarrolló un programa ejemplo

para el microprocesador. Desde este programa ejemplo se mandaban las instrucciones necesarias para configurar los módulos y conseguir las salidas deseadas.

Para analizar si los resultados obtenidos eran correctos, se empleó un osciloscopio en la medida de las señales generadas. Se observó que los resultados no eran siempre los deseados en el caso de los LEDs.

Tras analizar las posibles causas del fallo, primero se descartó la posibilidad de fallo por *hardware*. Teniendo en cuenta que en las simulaciones no se producían estos problemas, se concluyó que existía un problema de sincronismo entre el microprocesador y la FPGA. Realizando los cálculos correspondientes a los tiempos de ejecución de ambos dispositivos, desafortunadamente no se llegó a poder depurar el fallo de sincronismo, pero es algo en lo que se seguirá trabajando.

Finalmente, en relación con el **quinto objetivo**, se ha planteado un subconjunto de datos y sentencias de C para la descripción *hardware*. Para demostrar la viabilidad de este planteamiento, se han realizado dos pruebas en el IDE de Arduino en las que se han descrito en C los dos módulos que se tenían en VHDL (PWM y LEDs). Al emular el diseño, se vio que las señales generadas eran correctas.

Después de realizar los mismos diseños en VHDL y C, se ha demostrado la facilidad y sencillez del segundo con respecto al primero. La diferencia entre dificultades se ve reflejada en la cantidad de código necesaria para describir en un lenguaje lo mismo que en el otro (Figura 6.1).

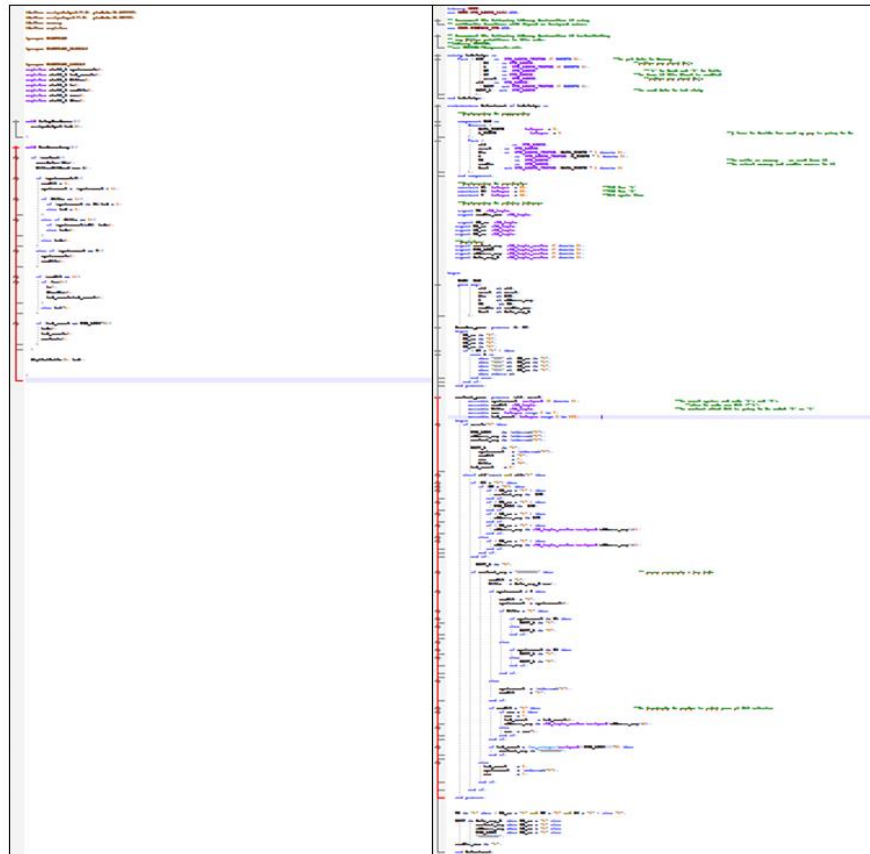


Figura 6.1- Módulo LEDs descrito en C (izquierda) y en VHDL (derecha). (Fuente: Autoría propia)

En definitiva, se puede decir que se han logrado los objetivos planteados en este estudio de acercamiento de dispositivos *hardware* tipo FPGAs a usuarios no expertos. Se espera que los resultados sean útiles para los futuros proyectos que continúen en esta línea de trabajo.

6.2. Líneas Futuras

Como ya se mencionó en capítulos anteriores, este trabajo es una primera aproximación a la propuesta de una metodología de diseño para dispositivos tipo FPGAs desde lenguaje C. Se pretende que trabajos posteriores utilicen los resultados de este estudio para llevar a cabo los siguientes pasos necesarios hasta conseguir el objetivo final de crear una plataforma de diseño digital para FPGAs orientada al “Do It Yourself”.

Las siguientes fases serían:

1. Diseñar una herramienta de traducción y síntesis de C a VHDL para que el programa escrito por el usuario se pudiese programar en la FPGA o dispositivo similar.
2. Crear un entorno de desarrollo *user-friendly* y sencillo donde describiesen los usuarios su diseño en lenguaje C. También se plantea la posibilidad de poder usar el IDE de Arduino, incorporando algunos cambios.
3. Crear un conjunto de ejemplos de módulos de complejidad creciente que el usuario pudiese incorporar a su diseño o utilizarlos en su formación.
4. Establecer una metodología docente para facilitar el aprendizaje del público de esta nueva herramienta.

7. Organización del TFG

7. Organización del TFG

7.1. Planificación temporal

En este capítulo se trata de mostrar la distribución temporal que se ha llevado a cabo para la realización del presente trabajo fin de grado. Para una mejor y fácil comprensión, se muestra mediante el apoyo de la estructura de descomposición del proyecto (EDP) (Figura 7.1) y el Diagrama de Gantt (Figura 7.2).

7.1.1. Estructura de Descomposición del Trabajo (EDP)

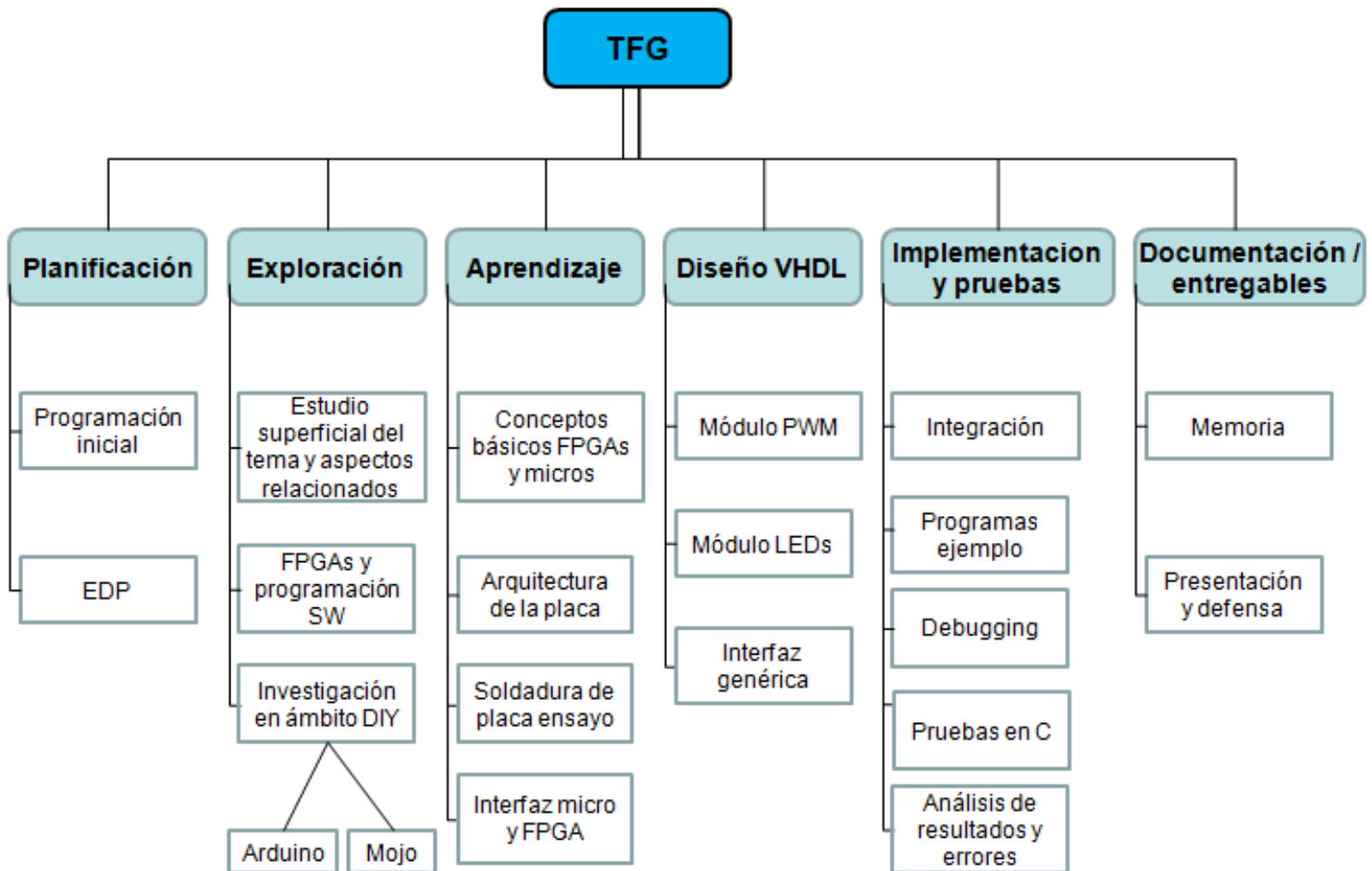


Figura 7.1- EDP (Fuente: Autoría propia)

7.1.2. Diagrama de Gantt



Figura 7.2 - Diagrama de Gantt del presente TFG. (Fuente: Autoría propia)

7.2. Presupuesto

En este apartado se ha realizado una estimación del coste total del proyecto. En la Tabla 7.1 se muestran los costes que se han considerado.

Para el cálculo de los costes asociados a las horas de trabajo de los implicados en el proyecto, se ha considerado el sueldo del alumno como el de un ingeniero junior y se ha calculado el coste por hora. De la misma manera se ha considerado que el sueldo de los tutores sea el de un ingeniero senior.

En cuanto a los gastos de material para la fabricación de la placa, se han tomado datos reales de catálogos de estos dispositivos.

Para las herramientas informáticas, electrónicas y de soldadura, se ha calculado su amortización teniendo en cuenta su vida útil y considerando el uso de estos equipos a lo largo de este año. Los años de vida útil tomados son de 5 años para el ordenador, 10 años para el osciloscopio, 5 años para el soldador y finalmente 6 años para la licencia del programa Xilinx ISE.

	Precio Unitario (€/Ud)	Cantidades (Uds)	Total (€)
Dedicación Alumno	10,00 €	376	3.760,00 €
Dedicación Tutor 1	30,00 €	18	540,00 €
Dedicación Tutor 2	30,00 €	35	1.050,00 €
Ordenador Dell Studio XPS 8100	800,00 €	0,20	160,00 €
Licencia Xilinx (1 año)	1.175,00 €	0,17	199,75 €
FPGA XC3S200 de Xilinx	20,00 €	1	20,00 €
Micro ATmega644PA de Atmel	10,00 €	1	10,00 €
Otros componentes de fabricación	20,00 €	1	20,00 €
Osciloscopio	800,00 €	0,10	80,00 €
Soldador	250,00 €	0,20	50,00 €
TOTAL			5.889,75 €

Tabla 7.1- Presupuesto del presente TFG. (Fuente: Autoría propia)

Referencias Bibliográficas

- [1] M. Wolf y S. McQuitty, "Understanding the Do-It-Yourself Consumer: DIY Motivation and Outcomes", *AMS Rev* [Internet] Available from: <http://www.diy-researchers.com/resources/>, 2011.
- [2] Fritzing, Fritzing, 2010. [En línea]. Available: <http://fritzing.org/home/> . [Último acceso: 2016].
- [3] O. G., «Arduino,» GNO, INC, 29 Agosto 2004. [En línea]. Available: <https://www.arduino.cc/>. [Último acceso: Marzo 2016].
- [4] Papilio, 2011. [En línea]. Available: <http://papilio.cc/> . [Último acceso: 2016].
- [5] R. J., Embedded Micro, 2003. [En línea]. Available: <https://embeddedmicro.com/>. [Último acceso: 2016].
- [6] N. J., «FPGA4FUN,» 2003. [En línea]. Available: <http://www.fpga4fun.com/> . [Último acceso: 2016].
- [7] U. E., « Raspberry Pi,» Raspberry Pi Foundation, 2008. [En línea]. Available: <https://www.raspberrypi.org/> . [Último acceso: 2016].
- [8] «[<https://opensource.org/osd>],» [En línea].
- [9] D. J. M. S. R. S. S. Banerjee, «Analytical and Comparative Evaluation of Advanced Digital PWM Control Techniques,» *IEEE*, 2014.
- [10] H.-G. Yang, «Overview: Emerging Technologies on Giga-Scale FPGA Implementation,» *IEEE*, 2010.
- [11] S. Trimberger, «Three Ages of FPGAs: A Retrospective on the First Thirty Years of FPGA Technology,» *IEEE*, vol. 103, nº 3, 2015.
- [12] F. H. Dirk Koch, *FPGAs for Software Programmers*, Springer.
- [13] M. M. D. D. G. A. T. Philippe Coussy, «An Introduction to High-Level Synthesis,» *IEEE*, 2009.
- [14] G. S. Gran Martin, «High-Level Synthesis: Past, Present, and Future,» *IEEE*, 2009.
- [15] Adafruit, *Datasheet WS2812/WS2818B*.

- [16] X. L. Q. W. C. T. Fuming Sun, «FPGA-based Embedded System Design,» *IEEE*, 2008.
- [17] S. W. K. Giang Nguyen, «Support of Cross Calls between a Microprocessor and FPGA in CPU-FPGA Coupling Architectures,» *IEEE*, 2010.

Índice de Figuras

<i>Figura 0.1- Planteamiento del flujo de diseño de microprocesador y FPGA integrados en una misma placa. (Fuente: Autoría propia)</i>	2
<i>Figura 0.2- Esquema general de placa con microcontrolador y FPGA. (Fuente: Autoría propia)</i>	3
<i>Figura 1.1- Placa Arduino UNO. (Fuente: [3])</i>	12
<i>Figura 1.2- Placa Mojo V3 (Fuente: [5])</i>	14
<i>Figura 1.3- Mojo IDE (Fuente: [5])</i>	14
<i>Figura 2.1- Esquema general de una placa con microprocesador y FPGA. (Fuente: Autoría propia)</i>	17
<i>Figura 3.1- Bloques lógicos configurables de una FPGA. (Fuente: [10])</i>	21
<i>Figura 3.2- Evolución de las características de las FPGAs. (Fuente: [11])</i>	22
<i>Figura 3.3- Flujo de diseño de una FPGA. (Fuente: [12])</i>	25
<i>Figura 3.4- Pasos de HLS. (Fuente: [13])</i>	26
<i>Figura 4.1- Parámetros de una señal PWM (Fuente: Autoría propia)</i>	30
<i>Figura 4.2- Generación de una señal PWM (Fuente: [5])</i>	31
<i>Figura 4.3- LEDs WS2812 (izquierda) y WS2812B (derecha) (Fuente: [15])</i>	32
<i>Figura 4.4- Composición de 24 bits de colores de un LED. (Fuente: [15])</i>	33
<i>Figura 4.5- Método de comunicación en cascada entre módulos LED. (Fuente: [15])</i>	33
<i>Figura 4.6- Método de transmisión de datos. (Fuente: [15])</i>	34
<i>Figura 4.7- Codificación de bits empleada en el protocolo (Fuente: [15])</i>	34
<i>Figura 4.8 - Esquema módulo LEDs. (Fuente: Autoría propia)</i>	36
<i>Figura 4.9- Esquema módulo interfaz genérica. (Fuente: Autoría propia)</i>	38
<i>Figura 4.10- Esquema del módulo interfaz genérica. (Fuente: Autoría propia)</i>	39
<i>Figura 4.11- Configuración PWM. (Fuente: Autoría propia)</i>	41
<i>Figura 4.12- Salidas PWM. (Fuente: Autoría propia)</i>	42
<i>Figura 4.13- Salida PWM con cambio de parámetros. (Fuente: Autoría propia)</i>	42
<i>Figura 4.14- Salida LEDs. (Fuente: Autoría propia)</i>	43
<i>Figura 4.15- Salida a la tira de LEDs ampliada. (Fuente: Autoría propia)</i>	43
<i>Figura 4.16- Flujo de diseño de microprocesador y FPGA. (Fuente: [16])</i>	44
<i>Figura 5.1- Ejemplo de estructura propuesta. (Fuente: Autoría propia)</i>	51
<i>Figura 5.2- Ejemplo de creación del componente en VHDL desde C. (Fuente: Autoría propia)</i>	52
<i>Figura 5.3- Ejemplo de declaración de las variables globales en VHDL desde C. (Fuente: Autoría propia)</i>	52
<i>Figura 5.4- Ejemplo de declaración de las variables locales en VHDL desde C. (Fuente: Autoría propia)</i>	53
<i>Figura 5.5- Ejemplo de traducción del HardwareLoop. (Fuente: Autoría propia)</i>	53
<i>Figura 5.6- PWM con duty cycle=128 (arriba) y duty cycle=8 (abajo). (Fuente: Autoría propia)</i>	55
<i>Figura 5.7- PWM con duty cycle=16 (arriba) y PWM con actualización de su duty cycle en cada período (abajo). (Fuente: Autoría propia)</i>	55
<i>Figura 5.8- Secuencia de bits modulados para controlar tiras de LEDs. (Fuente: Autoría propia)</i>	55
<i>Figura 6.1- Módulo LEDs descrito en C (izquierda) y en VHDL (derecha). (Fuente: Autoría propia)</i>	61
<i>Figura 7.1- EDP (Fuente: Autoría propia)</i>	65
<i>Figura 7.2 - Diagrama de Gantt del presente TFG. (Fuente: Autoría propia)</i>	66

Índice de Tablas

Tabla 1.1 - Proyectos DIY destacados en el ámbito de la electrónica (Fuente: Autoría Propia) ¡Error! Marcador no definido.

Tabla 4.1- Especificaciones de tiempos de los LEDs WS2812 y WS2812B (Fuente:Autoría propia) 35

Tabla 7.1- Presupuesto del presente TFG. (Fuente: Autoría propia)..... 67

Abreviaturas y Acrónimos

bps	bits per second
CLB	Configurable Logic Block
CPU	Central Processing Unit
DIE	División de Ingeniería Electrónica
DIY	Do It Yourself
EDA	Electronic design automation
EDP	Estructura de Descomposición del Proyecto
ETSII	Escuela Técnica Superior de Ingenieros Industriales
FPGA	Field Programmable Gate Array
HDL	Hardware Description Language
HLS	High Level Synthesis
HW	Hardware
IC	Integrated Chip
IDE	Integrated Development Environment
ISE	Integrated Software Environment
JTAG	Joint Test Action Group
LED	Light Emitting Diode
MMU	Memory Management Unit
PCB	Printed Circuit Board
PWM	Pulse-Width Modulation
RGB	Red-Green-Blue
RTL	Register Transfer Level
SW	Software
TFG	Trabajo Fin de Grado
UPM	Universidad Politécnica de Madrid
USB	Universal Serial Bus
VHDL	VHSIC Hardware Description Language
VHSIC	Very High Speed Integrated Circuit

